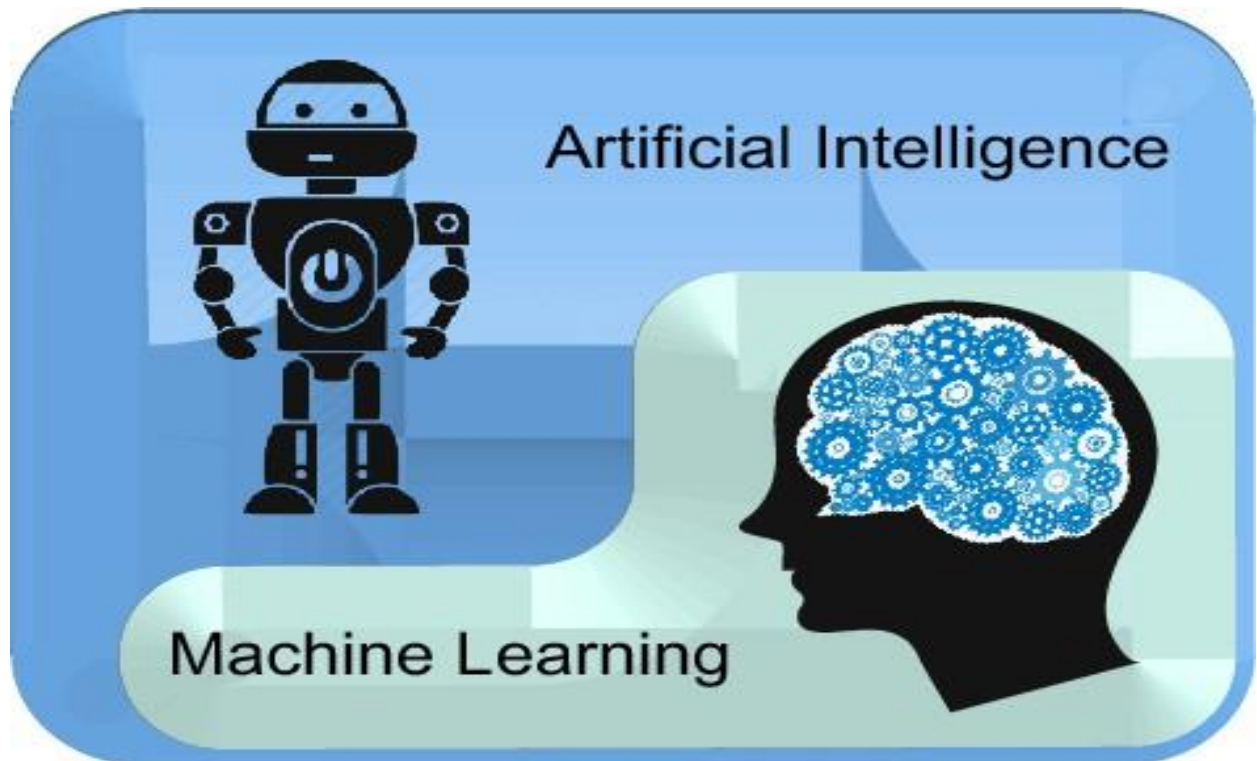


DAYANANDA SAGAR UNIVERSITY
SCHOOL OF ENGINEERING

DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING
Artificial Intelligence & Machine Learning



Database Management Systems Laboratory Manual
24AM2403



Dayananda Sagar University



SCHOOL OF
ENGINEERING

Dayananda Sagar University

School of Engineering



Devarakaggalahalli, Harohalli Kanakapura Road, Dt, Ramanagara, Karnataka 562112.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(Artificial Intelligence & Machine Learning)

Vision and Mission

Vision

To be a centre of excellence in education, research & training, innovation & entrepreneurship and to produce citizens with exceptional leadership qualities to serve national and global needs.

Mission

To achieve our objectives in an environment that enhances creativity, innovation and scholarly pursuits while adhering to our vision.

Values

The Pursuit of Excellence

A commitment to strive continuously to improve ourselves and our systems with the aim of becoming the best in our field.

Fairness

A commitment to objectivity and impartiality, to earn the trust and respect of society.

Leadership

A commitment to lead responsively and creatively in educational and research processes.

Integrity and Transparency

A commitment to be ethical, sincere and transparent in all activities and to treat all individuals with dignity and respect.



SCHOOL OF
ENGINEERING

Dayananda Sagar University

School of Engineering



Devarakaggalahalli, Harohalli Kanakapura Road, Dt, Ramanagara, Karnataka 562112.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(Artificial Intelligence & Machine Learning)

Vision

- To produce graduates in Computer Science and Engineering (Artificial Intelligence & Machine Learning) through excellence in education and research with an emphasis on sustainable eco-system that contributes significantly to the society.

Mission

The Department Computer Science and Engineering (Artificial Intelligence & Machine Learning) is committed to:

- Impart quality education through the state-of-the-art curriculum, infrastructure facilities, cutting edge technologies, Sustainable learning practices and lifelong learning.
- Collaborate with industry-academia and inculcate interdisciplinary research to transform professionals into technically competent.
- Produce engineers and techno-entrepreneurs for global needs.



SCHOOL OF
ENGINEERING

Dayananda Sagar University

School of Engineering



Devarakaggalahalli, Harohalli Kanakapura Road, Dt, Ramanagara, Karnataka 562112.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence & Machine Learning)

Department PEO's, PO's and PSO's :

- PEO1. Apply appropriate theory, practices, and tools of machine intelligence to the specification, design, implementation, maintenance, and evaluation in the workplace or in higher education.
- PEO2. Adapt, contribute and innovate new technologies in their computing profession by working in teams to design, implement, and maintain in the key domains of Artificial Intelligence & Machine Learning.
- PEO3. Function effectively in the work place as competent Artificial Intelligence & Machine Learning Professionals, Entrepreneurs or Researchers or maintain employment through lifelong learning such as professional conferences, certificate programs or other professional educational activities, ethics, and societal awareness.

Programme Outcome (PO's)

- PO1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- PO2. Problem analysis: Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

- PO3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- PO4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- PO5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
- PO6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- PO7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- PO8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- PO9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- PO10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- PO11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member

and leader in a team, to manage projects and in multidisciplinary environments.

- PO12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes (PSO's)

Engineering graduates will be to:

- PSO1. Cognitive Outcome: Solve complex engineering problems in computing by applying the principles in Artificial Intelligence, Machine Learning, Network Engineering, Software Engineering, Data Engineering and Intelligent Systems.
- PSO2. Skill & Design Outcome: Apply technical skills and research skills through professional societies, certification programs, projects, internships and laboratory exercises to design & develop algorithms, programs, and projects using modern software tools to provide the sustainable solutions to Computer Science, Artificial Intelligence and Machine Learning problems related to the society and environment.



Perquisite Courses (if any)			
#	Sem / Year	Course Code	Title of the Course
1	III/II	22CS2305	COMPUTATIONAL THINKING WITH PYTHON
2	IV/II	22AM2401	Artificial Intelligence -I

COURSE OBJECTIVES:

1. Acquire the concept of databases, Entity-Relationship Model and relational model for creating and designing databases for the real-world scenario.
2. Develop queries to extract data from the databases using a structured query language.
3. Differentiate SQL and NoSQL.
4. Demonstrate the operations on MongoDB, Database connectivity with front end and Optimize the Database design using Normalization Concepts.
5. Understand the importance of Transaction Management, Concurrency control mechanism and recovery techniques.

COURSE OUTCOMES:

CO No.	Outcomes	Bloom's TaxonomyLevel
1	Use the basic concepts of database management system in the design and creating database blueprint using E-R model and relational model.	L3
2	Formulate SQL and NOSQL queries for building structure and unstructured databases	L3
3	Demonstrate database connectivity using vendor specific drivers	L3
4	Apply normalization techniques to design relational database management system	L3
5	Adapt Transactional Management, Concurrency control and recovery management techniques in database management system.	L3

Table: Mapping Levels of COs to POs / PSOs														
COs	Program Outcomes (POs)												PSOs	
	1	2	3	4	5	6	7	8	9	10	11	12	1	2
CO1	3	3	2	-	-	-	-	-	2	2	-	2	3	-
CO2	3	2	1	-	3	-	-	-	2	2	-	2	3	-
CO3	2	2	2	-	3	-	-	-	2	2	-	2	3	-
CO4	3	1	2	-	1	-	-	-	2	2	-	2	3	-
CO5	2	1	-	-	-	-	-	-	2	2	-	2	3	-



Instructions for Laboratory Exercises:

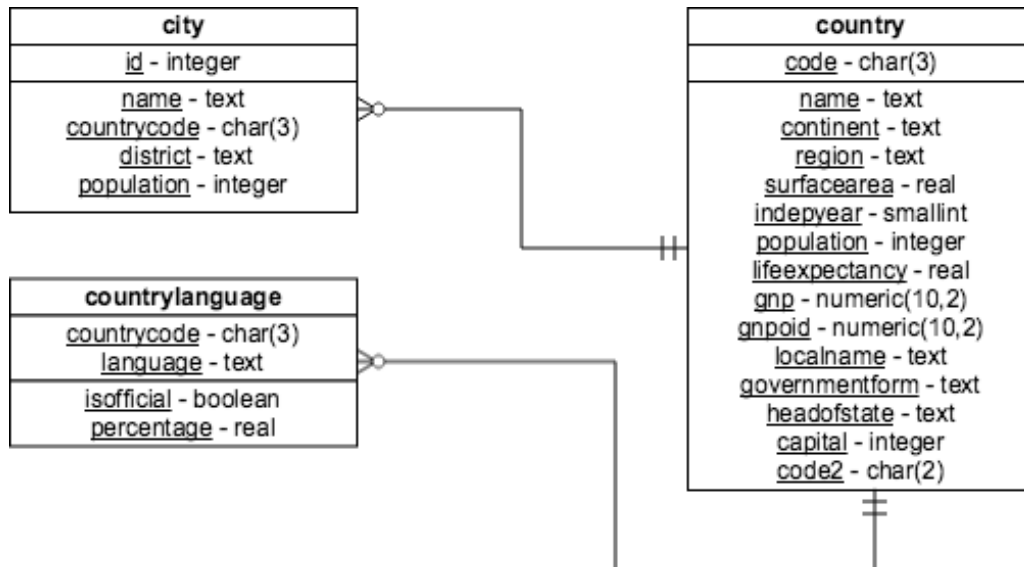
1. Launch MySQL Workbench to open the home screen. Existing connections are shown when you click the MySQL Connections view from the sidebar. No connections exist for first-time users.
2. To add a connection, click the [+] icon to the right of the MySQL Connections title on the [home](#) screen. This opens the Setup New Connection. Define the Connection Name value,
3. Read the Configure Local Management introduction (shown in the next figure), and press Next to begin defining the new connection parameters.
4. The connection will now be tested. You should see that the connection was successful. If not, click Back and check that you have entered the information correctly.

List of Experiments

Exp No	Experiment Name	Date	Marks	Sign
1	Design any database with at least 3 entities and establish proper relationships between them. Draw suitable ER/EER diagrams for the system. Apply DCL and DDL commands.			
2	Design and implement a database and apply at least 10 Different DML Queries for the following task. a. For a given input string display only those records which match the given pattern or a phrase in the search string. Make use of wild characters and like operators for the same. Make use of Boolean and arithmetic operators wherever necessary			
3	Write SQL statements to join table and retrieve the combined information from tables.			
4	Execute the Aggregate functions count, sum, avg, min, max on a suitable database. Make use of built in functions according to the need of the database chosen.			
5	Retrieve the data from the database based on time and date functions like now(), date(), day(), time() etc., Use of group by and having clauses.			
6	Write and execute database trigger. Consider row level and statement level triggers.			
7	Write and execute program to perform operations on MongoDB Database.			
8	Write and execute program to perform CRUD operations.			
Open Ended Experiments				
9	Consider the Table “employees”, write a SQL query to remove all the duplicate emails of employees keeping the unique email with the lowest employee id, return employee id and unique emails .			
10	A salesperson is a person whose job is to sell products or services. Consider the table “Sales” [given below]. Write a SQL query to find the top 10 salesperson that have made highest sale. Return their names and total sale amount.			

Internals (40M)	Lab Record & Attendance (20M)	Total Marks (60M)

World database tables and attributes



EXPERIMENT: 1

AIM: To Design any database with at least 3 entities and establish proper relationships between them and Draw suitable ER/EER diagrams for the system. Apply DCL, DDL and DML commands.

PROGRAM:

Consider the schema of WORLD database.

-- ER diagram of World database

-- DCL commands

===== DCL =====

-- Grant and Revoke

-- select user from mysql.user;

select user from mysql.user;

-- show grants for 'user@localhost';

show grants for 'mysql.session'@'localhost';

-- grant all previleages on *.* to `mysql.infoschema`@`localhost`;

grant ALL PRIVILEGES on *.* to 'root'@'localhost';

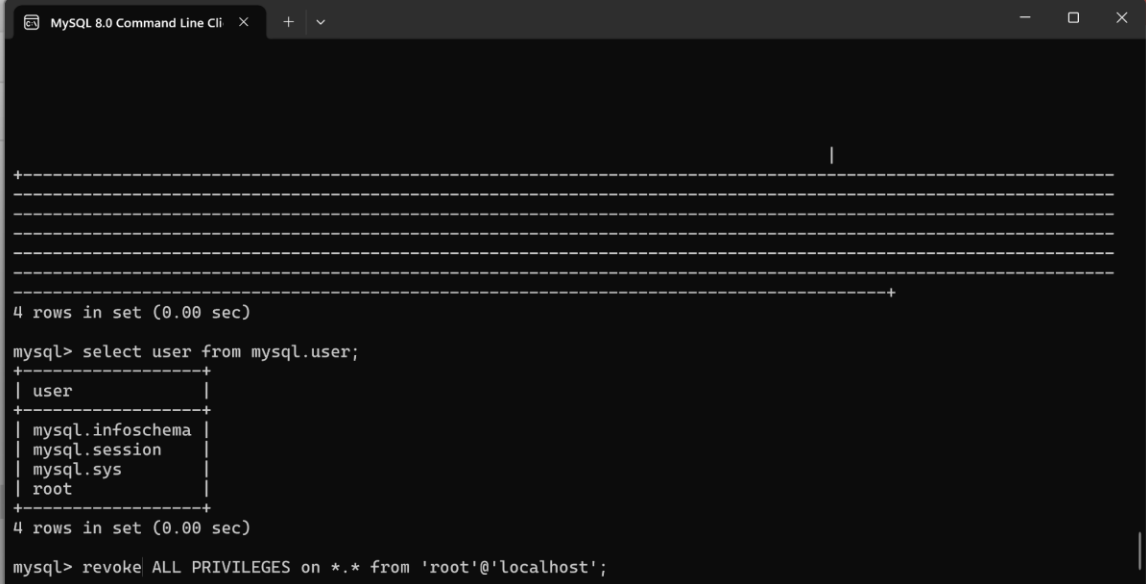
-- revoke all previleages on *.* from `mysql.infoschema`@`localhost`--

revoke;

revoke ALL PRIVILEGES on *.* from 'root'@'localhost';

-- DDL commands

-- Create drop alter rename create index drop index modify



```
MySQL 8.0 Command Line Cli  x  +  v

+-----+
| user |
+-----+
| mysql.infoschema |
| mysql.session |
| mysql.sys |
| root |
+-----+
4 rows in set (0.00 sec)

mysql> select user from mysql.user;
+-----+
| user |
+-----+
| mysql.infoschema |
| mysql.session |
| mysql.sys |
| root |
+-----+
4 rows in set (0.00 sec)

mysql> revoke ALL PRIVILEGES on *.* from 'root'@'localhost';
```

```
MySQL 8.0 Command Line Cli x + v

| GRANT APPLICATION_PASSWORD_ADMIN,AUDIT_ABORT_EXEMPT,AUDIT_ADMIN,AUTHENTICATION_POLICY_ADMIN,BACKUP_ADMIN,BINLOG_ADMIN,BINLOG_ENCRYPTION_ADMIN,CLONE_ADMIN,CONNECTION_ADMIN,ENCRYPTION_KEY_ADMIN,FIREWALL_EXEMPT,FLUSH_OPTIMIZER_COSTS,FLUSH_STATUS,FLUSH_TABLES,FLUSH_USER_RESOURCES,GROUP_REPLICATION_ADMIN,GROUP_REPLICATION_STREAM,INNODB_REDO_LOG_ARCHIVE,INNODB_REDO_LOG_ENABLE,PASSWORDLESS_USER_ADMIN,PERSIST_RO_VARIABLES_ADMIN,REPLICATION_APPLIER,REPLICATION_SLAVE_ADMIN,RESOURCE_GROUP_ADMIN,RESOURCE_GROUP_USER,ROLE_ADMIN,SENSITIVE_VARIABLES_OBSERVER,SERVICE_CONNECTION_ADMIN,SESSION_VARIABLES_ADMIN,SET_USER_ID,SHOW_ROUTINE,SYSTEM_USER,SYSTEM_VARIABLES_ADMIN,TABLE_ENCRYPTION_ADMIN,TELEMETRY_LOG_ADMIN,XA_RECOVER_ADMIN ON *.* TO 'root'@'localhost' |
| GRANT PROXY ON ''@'' TO 'root'@'localhost' WITH GRANT OPTION

+-----+
|
+-----+

3 rows in set (0.00 sec)

mysql> show grants for 'root'@'localhost';
```

```
MySQL 8.0 Command Line Cli x + v

| GRANT APPLICATION_PASSWORD_ADMIN,AUDIT_ABORT_EXEMPT,AUDIT_ADMIN,AUTHENTICATION_POLICY_ADMIN,BACKUP_ADMIN,BINLOG_ADMIN,BINLOG_ENCRYPTION_ADMIN,CLONE_ADMIN,CONNECTION_ADMIN,ENCRYPTION_KEY_ADMIN,FIREWALL_EXEMPT,FLUSH_OPTIMIZER_COSTS,FLUSH_STATUS,FLUSH_TABLES,FLUSH_USER_RESOURCES,GROUP_REPLICATION_ADMIN,GROUP_REPLICATION_STREAM,INNODB_REDO_LOG_ARCHIVE,INNODB_REDO_LOG_ENABLE,PASSWORDLESS_USER_ADMIN,PERSIST_RO_VARIABLES_ADMIN,REPLICATION_APPLIER,REPLICATION_SLAVE_ADMIN,RESOURCE_GROUP_ADMIN,RESOURCE_GROUP_USER,ROLE_ADMIN,SENSITIVE_VARIABLES_OBSERVER,SERVICE_CONNECTION_ADMIN,SESSION_VARIABLES_ADMIN,SET_USER_ID,SHOW_ROUTINE,SYSTEM_USER,SYSTEM_VARIABLES_ADMIN,TABLE_ENCRYPTION_ADMIN,TELEMETRY_LOG_ADMIN,XA_RECOVER_ADMIN ON *.* TO 'root'@'localhost' |
| GRANT PROXY ON ''@'' TO 'root'@'localhost' WITH GRANT OPTION

+-----+
|
+-----+

3 rows in set (0.00 sec)

mysql> grant ALL PRIVILEGES on *.* to 'root'@'localhost';
Query OK, 0 rows affected (0.00 sec)

mysql>
```

```
MySQL 8.0 Command Line Cli x + -
```

```
| GRANT APPLICATION_PASSWORD_ADMIN,AUDIT_ABORT_EXEMPT,AUDIT_ADMIN,AUTHENTICATION_POLICY_ADMIN,BACKUP_ADMIN,BINLOG_ADMIN,BINLOG_ENCRYPTION_ADMIN,CLONE_ADMIN,CONNECTION_ADMIN,ENCRYPTION_KEY_ADMIN,FIREWALL_EXEMPT,FLUSH_OPTIMIZER_COSTS,FLUSH_STATUS,FLUSH_TABLES,FLUSH_USER_RESOURCES,GROUP_REPLICATION_ADMIN,GROUP_REPLICATION_STREAM,INNODB_B_REDO_LOG_ARCHIVE,INNODB_REDO_LOG_ENABLE,PASSWORDLESS_USER_ADMIN,PERSIST_RO_VARIABLES_ADMIN,REPLICATION_APPLIER,REPLICATION_SLAVE_ADMIN,RESOURCE_GROUP_ADMIN,RESOURCE_GROUP_USER,ROLE_ADMIN,SENSITIVE_VARIABLES_OBSERVER,SERVICE_CONNECTION_ADMIN,SESSION_VARIABLES_ADMIN,SET_USER_ID,SHOW_ROUTINE,SYSTEM_USER,SYSTEM_VARIABLES_ADMIN,TABLE_ENCRYPTION_ADMIN,TELEMETRY_LOG_ADMIN,XA_RECOVER_ADMIN ON *.* TO 'root'@'localhost' |  
| GRANT PROXY ON ``@`` TO 'root'@'localhost' WITH GRANT OPTION  
  
+-----+  
+-----+  
+-----+  
+-----+  
+-----+  
+-----+  
+-----+  
+-----+  
+-----+  
+-----+
```

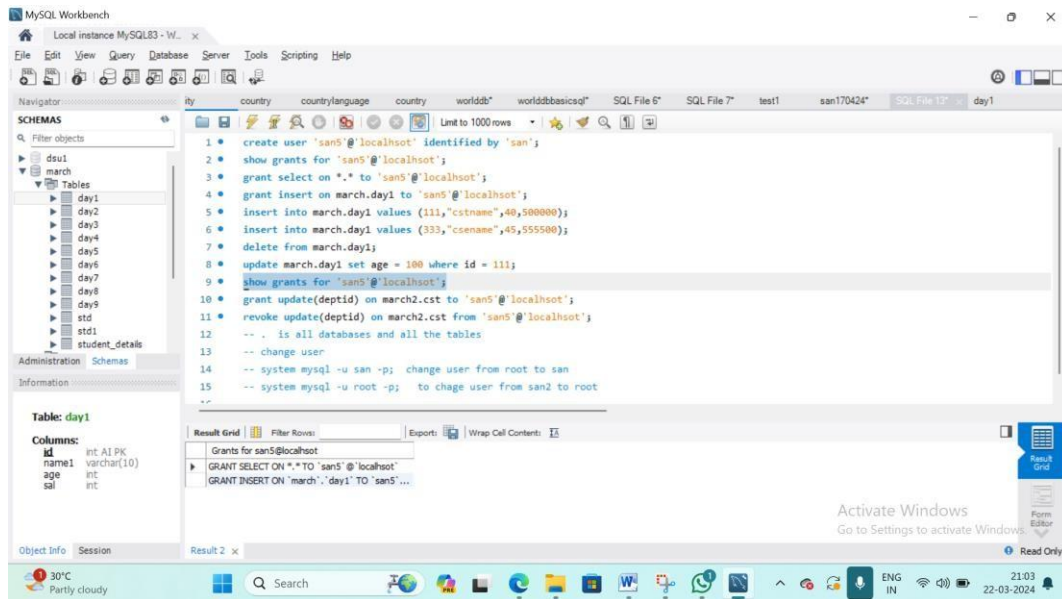
```
3 rows in set (0.00 sec)
```

```
mysql> grant ALL PRIVILEGES on *.* to 'root'@'localhost';  
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> revoke ALL PRIVILEGES on *.* from 'root'@'localhost';
```

81° Windows Search File Explorer Mail Edge Calendar Weather Settings Network & Internet ENG IN 10:36 AM 3/21/2024

```
===== DCL =====
create user 'san5'@'localhsot' identified by 'san';
show grants for 'san5'@'localhsot';
grant select on databasename.tablename to 'san5'@'localhsot';
grant select on *.* to 'san5'@'localhsot';
grant insert on march2.cst to 'san5'@'localhsot';
insert into march2.cst values (111,"cstname","3rd","a",500);
insert into march2.cst values (333,"cstname","3rd","a",500);
delete from march2.cst;
update march2.cst set deptgrade = "A+" where deptid = 111;
update march2.cst set deptid = 1111 where deptgrade = "A+";
grant update(deptid) on march2.cst to 'san5'@'localhsot';
revoke update(deptid) on march2.cst from 'san5'@'localhsot';
-- . is all databases and all the tables
-- change user
-- system mysql -u san -p; change user from root to san
-- system mysql -u root -p; to chage user from san2 to root
```



===== DDL =====

```

use cst;
create table std (
  id int,
    name1 varchar(100)
);
desc std;
alter table std
add placement varchar(100);

```

```

alter table std
modify column placement int;

```

```

desc std;
alter table std
drop placement;

```

```

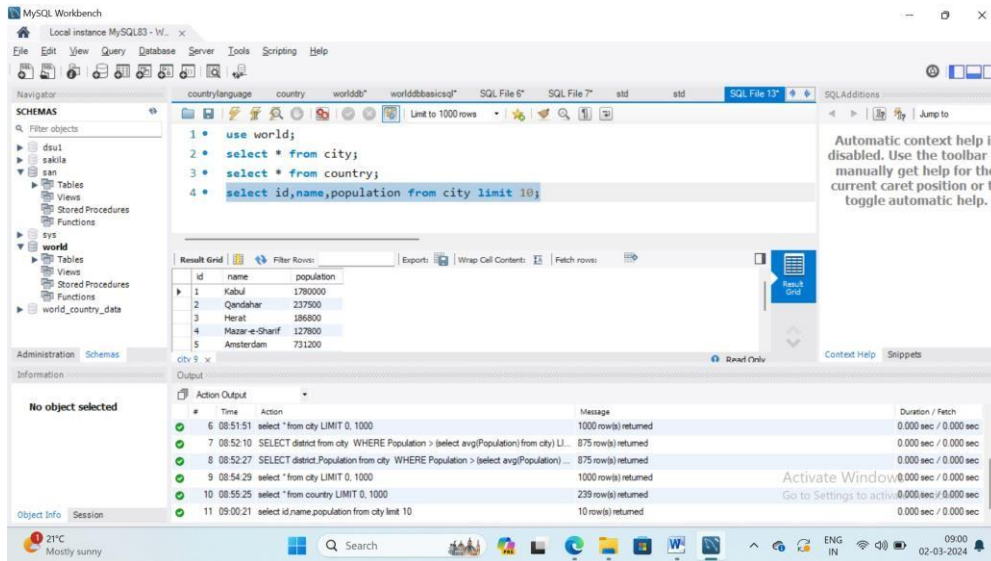
desc std;
alter table std
rename column name1 to name;

```

===== DML =====

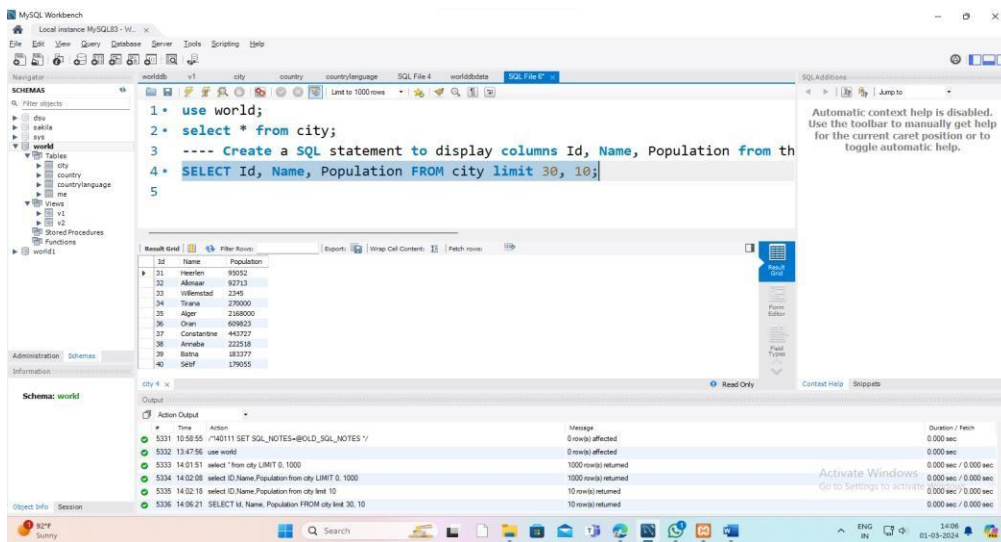
- a) Create a SQL statement to display columns Id, Name, Population from the city table and limit results to first 10 rows only.

Select Id,name,Population from city limit 10;



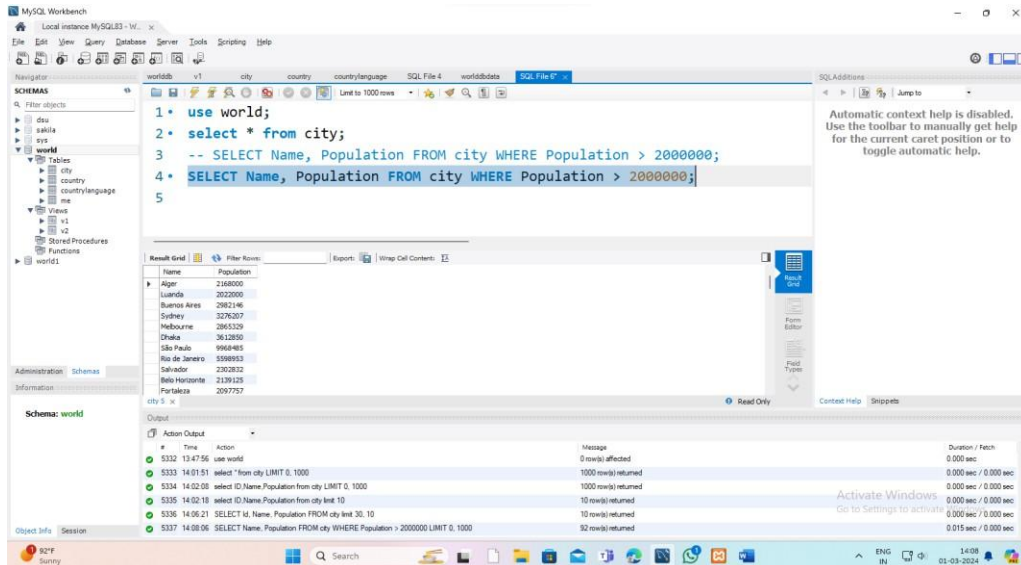
b) Create a SQL statement to display columns Id, Name, Population from the city table and limit results to rows 31-40

Select Id,Name, Population from city limit 30,10;



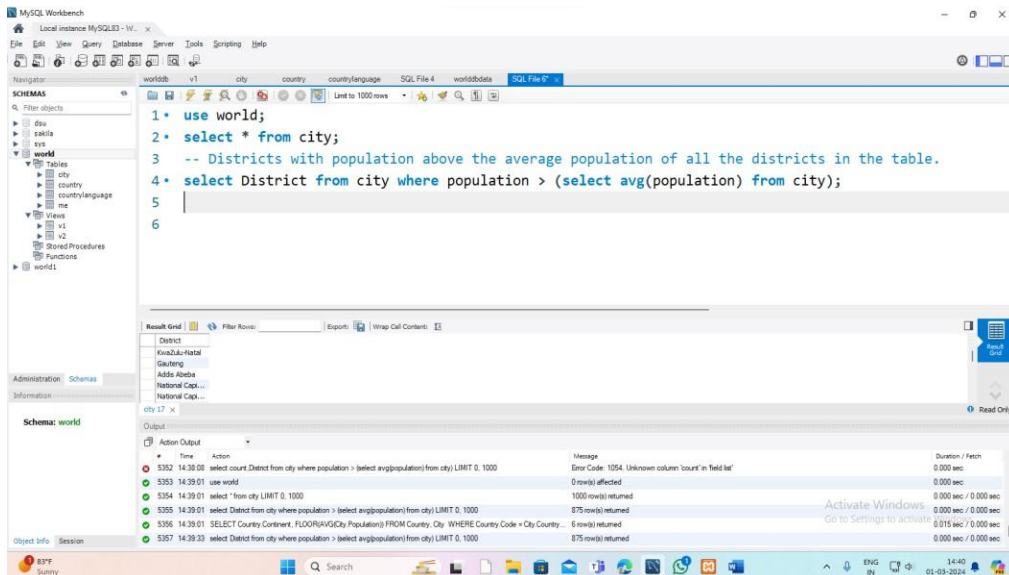
- c) Create a SQL statement to find only those cities from city table whose population is larger than 2000000.

Select Name, population from city where population >2000000



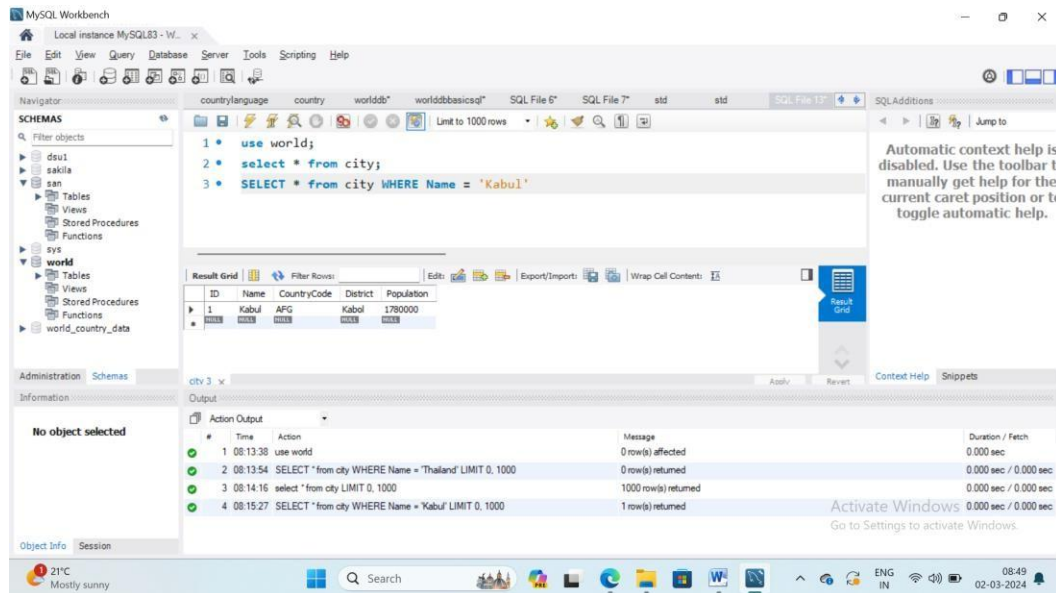
- d) Select the Districts with population above the average population of all the districts in the table.

Select district from city where population > (select avg(population) from city);



e) What is the country code for Kabul

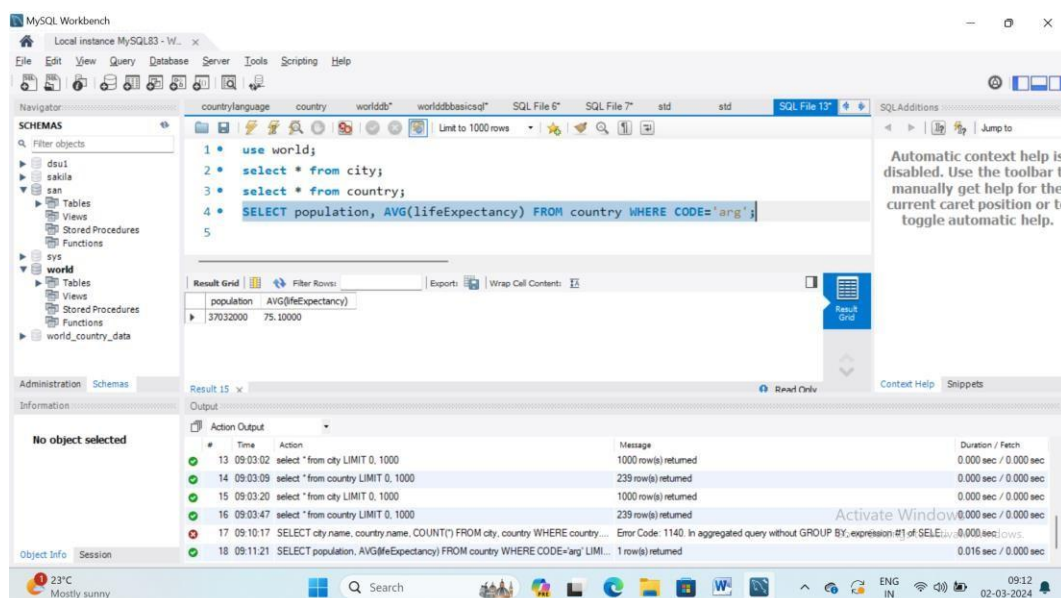
`SELECT * from city WHERE Name = 'Kabul'`



f) Find out what the population and average life expectancy for people in Argentina.

(ARG) is (37032000, 75.10000)

`SELECT population, AVG(lifeExpectancy) FROM country WHERE CODE='arg';`

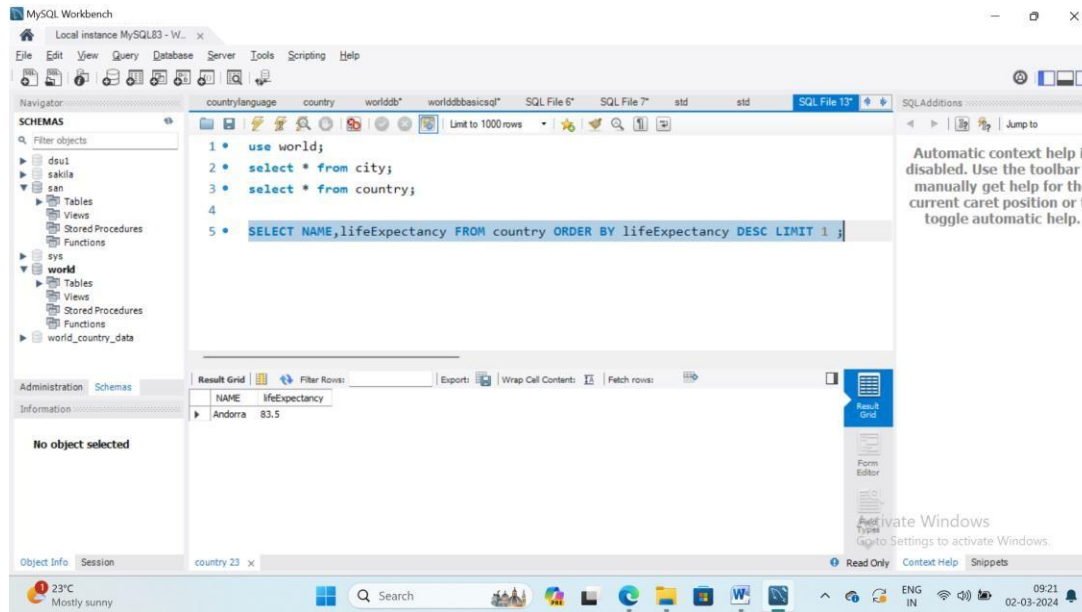


g) Using IS NOTNULL,ORDER BY,LIMIT,what country has the highest life expectancy?

(Andorra, 83.5)

```
SELECT * FROM country;
```

```
SELECT NAME,lifeExpectancy FROM country ORDER BY lifeExpectancy DESC  
LIMIT 1 ;
```



Extra code for practice =====

```
1.SELECT TOP 3 * FROM Country  
WHERE Continent='Africa';
```

```
2.SELECT * FROM countrylanguages  
WHERE Language IN ('Dutch', 'English', 'Ambo');
```

```
3. SELECT MIN(Percentage) FROM Countrylanguages;
```

```
4. SELECT * FROM City WHERE Population BETWEEN 100000 AND 200000.
```

```
5. INSERT INTO table_name (column1, column2, column3, ...)  
VALUES (value1, value2, value3, ...);
```

VIVA QUESTIONS

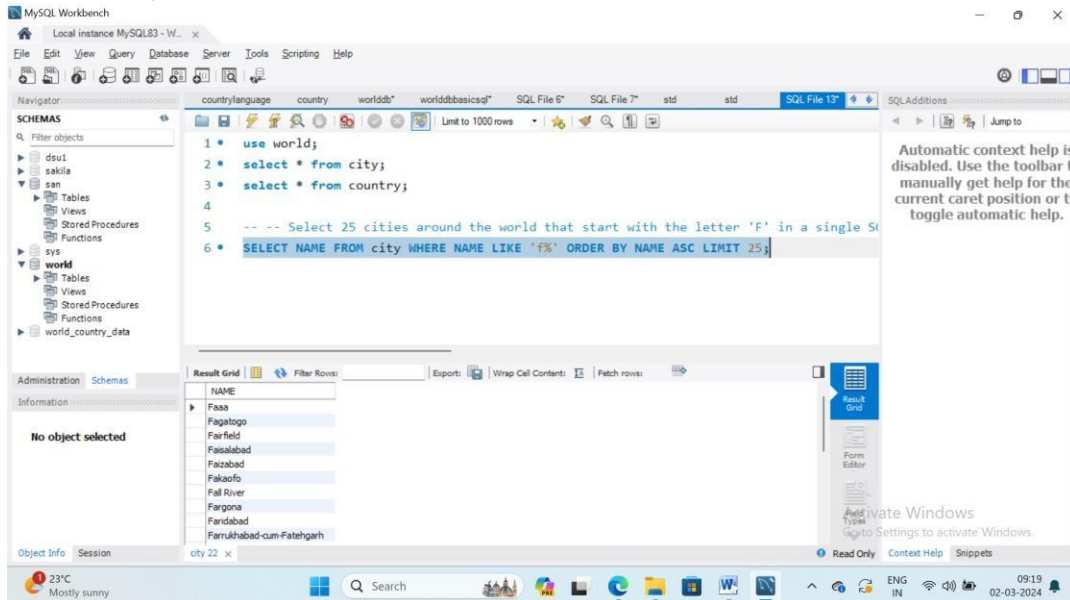
EXPERIMENT: 2

AIM: To write a SQL Query for Use of order by functions and LIKE function.

PROGRAM:

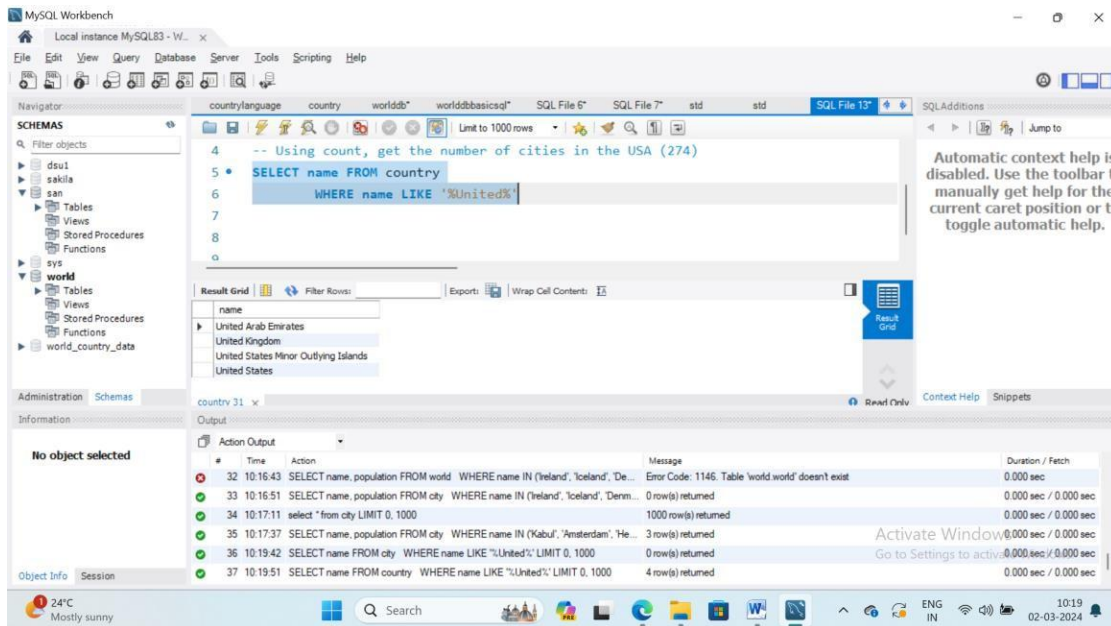
Select 25 cities around the world that start with the letter 'F' in a single SQL query.

```
SELECT NAME FROM city WHERE NAME LIKE 'f%' ORDER BY NAME ASC  
LIMIT 25;
```



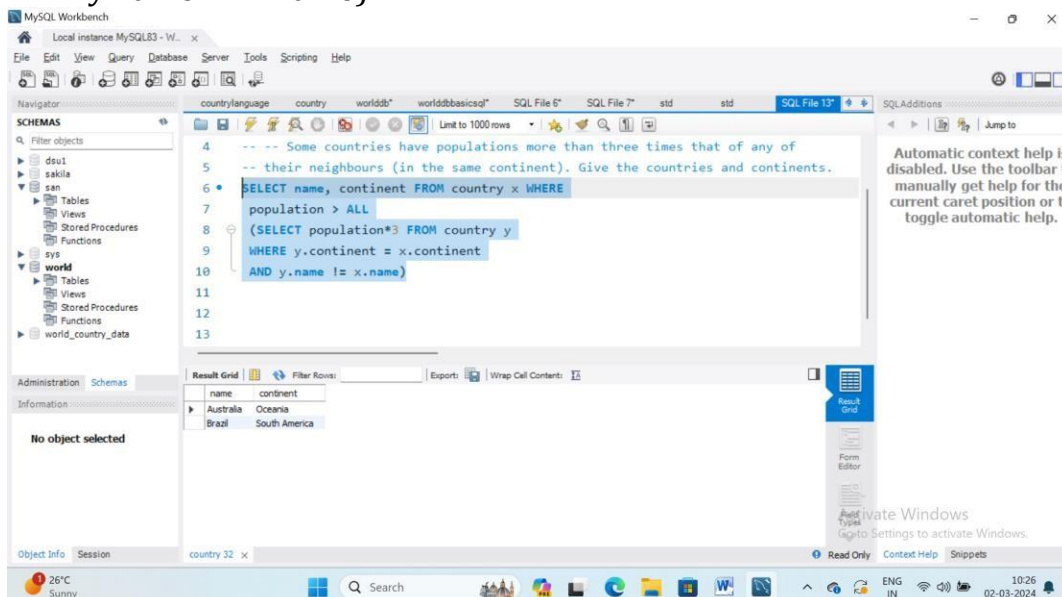
- a) Show the countries which have a name that includes the word 'United'

```
SELECT name FROM world  
WHERE name LIKE '%United%'
```



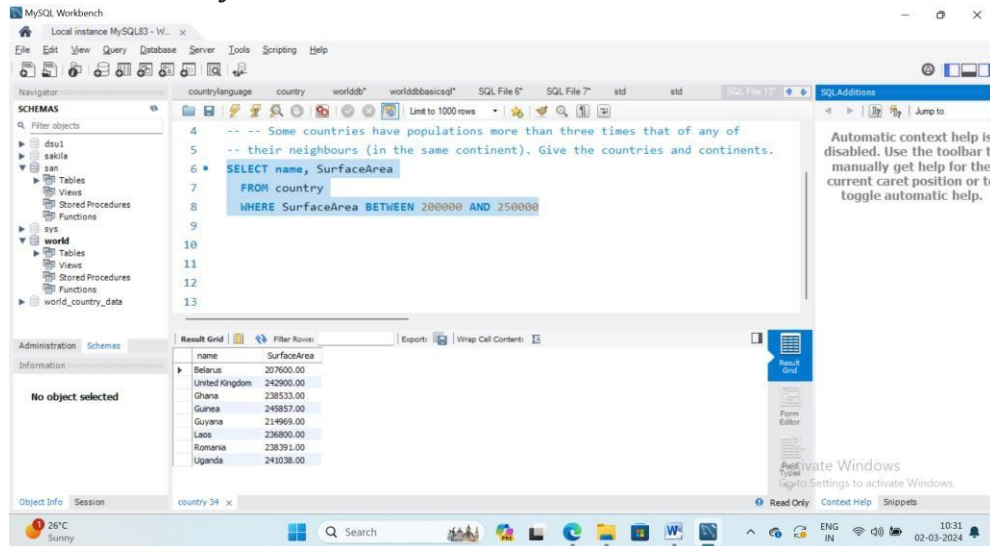
b) Some countries have populations more than three times that of any of their neighbours (in the same continent). Give the countries and continents.

```
SELECT name, continent FROM country x WHERE
population > ALL
(SELECT population*3 FROM country y
WHERE y.continent = x.continent
AND y.name != x.name)
```



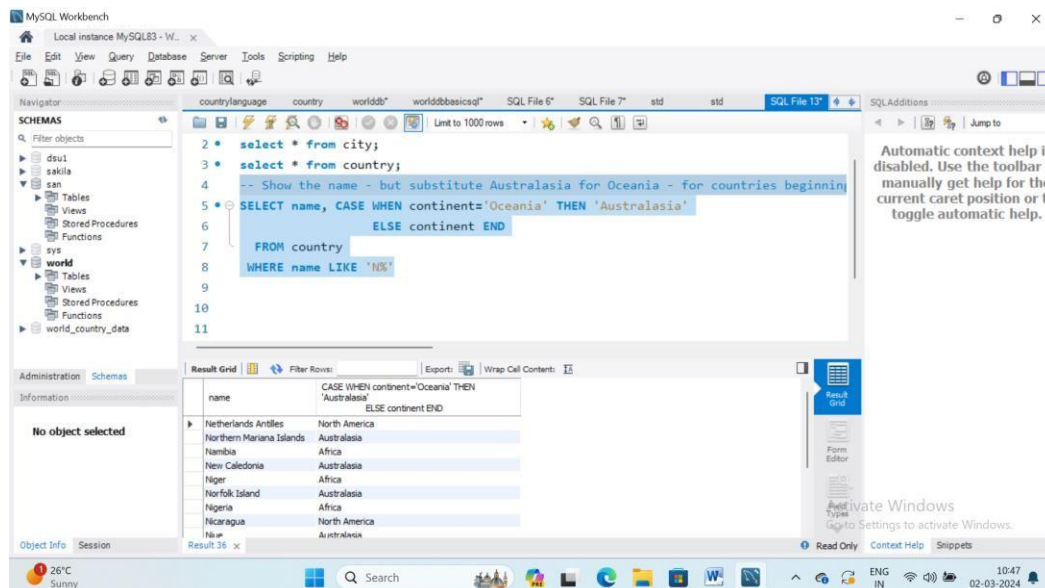
c) Give name and surface area from country where surface area between 200000 and 250000.

SELECT name, SurfaceArea
FROM country WHERE SurfaceArea BETWEEN 200000 AND 250000



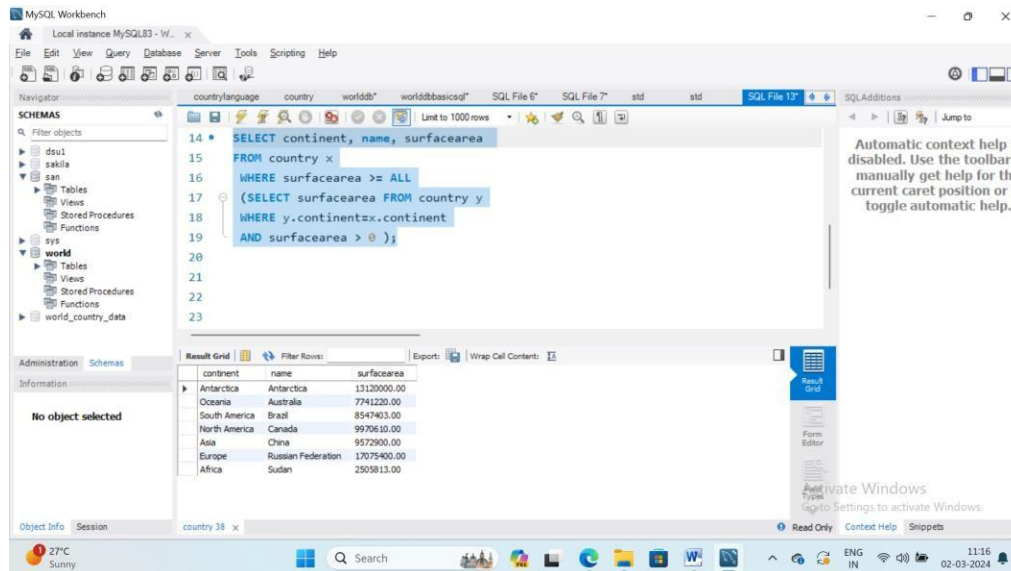
- d) Show the name - but substitute Australasia for Oceania - for countries beginning with N.

SELECT name, CASE WHEN continent='Oceania' THEN 'Australasia'
ELSE continent END
FROM country
WHERE name LIKE 'N%'



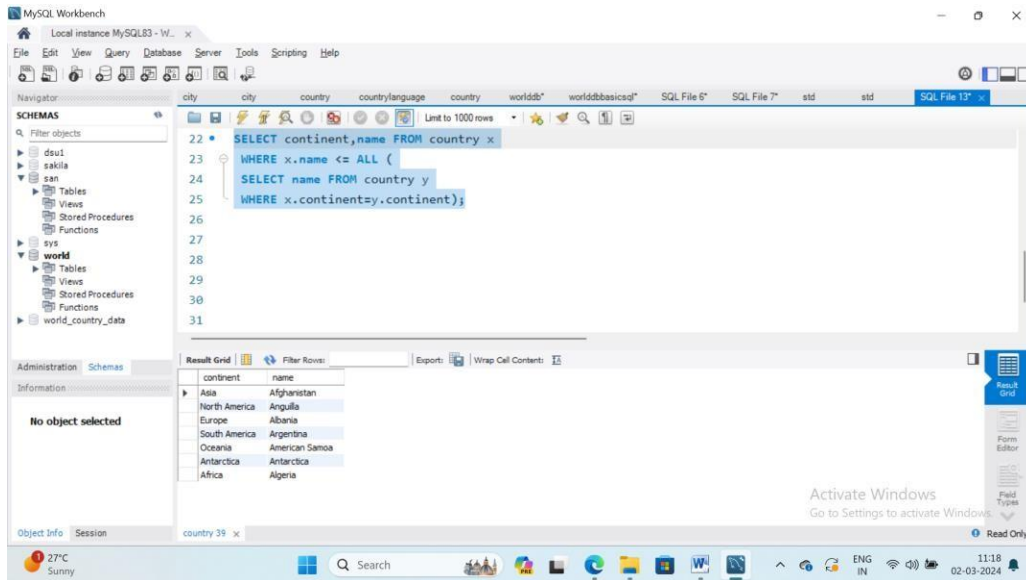
- e) Find the largest country (by area) in each continent, show the continent, the name and the area:

```
SELECT continent, name, surfacearea
FROM country x
WHERE surfacearea >= ALL
(SELECT surfacearea FROM country y
WHERE y.continent=x.continent
AND surfacearea > 0 );
```



- f) List each continent and the name of the country that comes first alphabetically

```
SELECT continent,name FROM country x
WHERE x.name <= ALL (
SELECT name FROM country y
WHERE x.continent=y.continent);
```

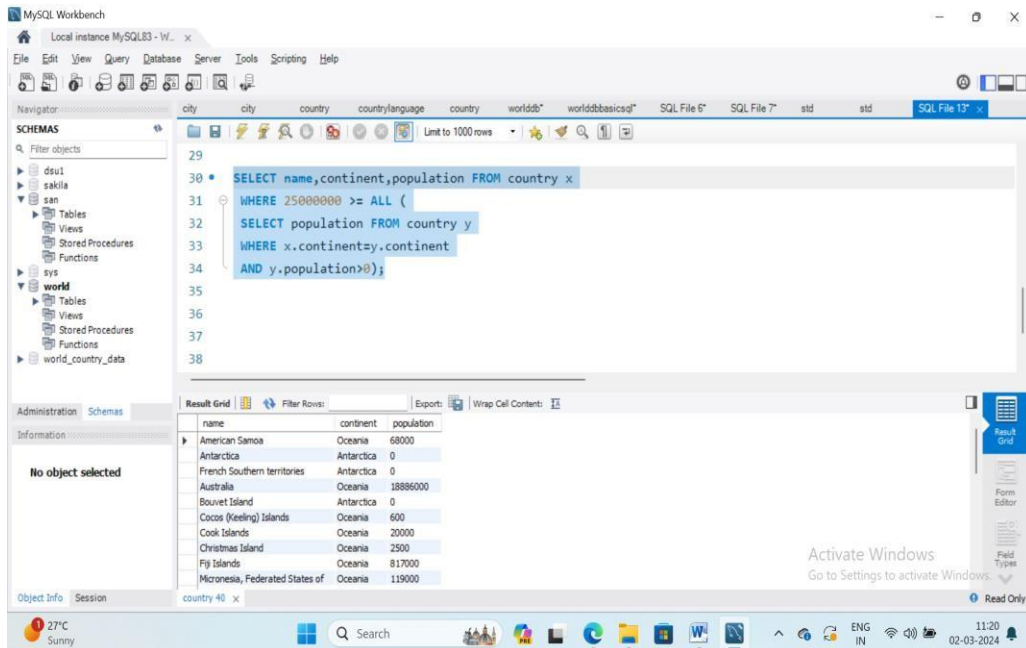


- g) Find the continents where all countries have a population ≤ 25000000 . Then find the names of the countries associated with these continents. Show name, continent and population.

```

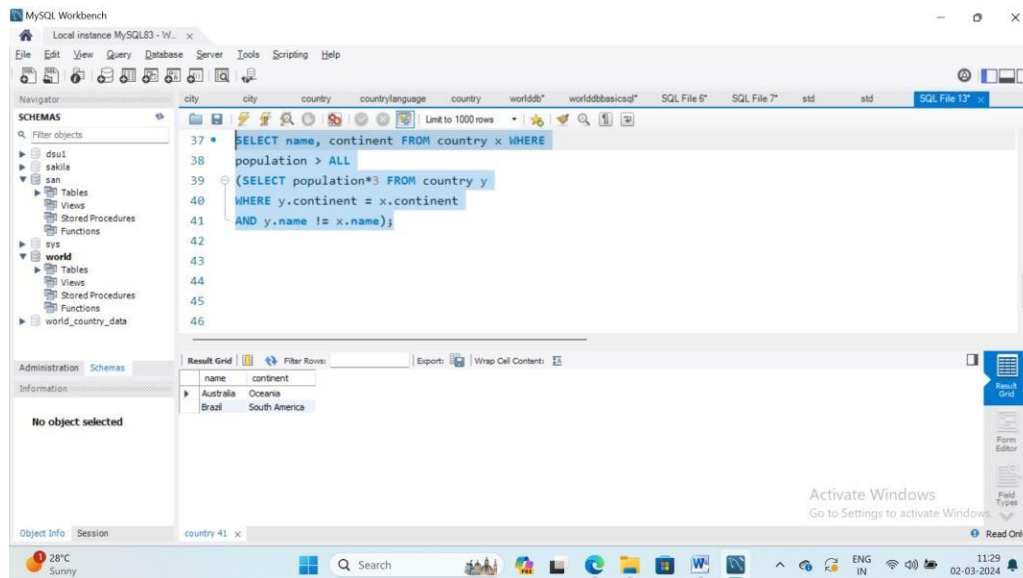
SELECT name,continent,population FROM country x
WHERE 25000000 >= ALL (
  SELECT population FROM country y
  WHERE x.continent=y.continent
  AND y.population>0);

```



- h) Some countries have populations more than three times that of any of their neighbours (in the same continent). Give the countries and continents.

```
SELECT name, continent FROM country x WHERE  
population > ALL  
(SELECT population*3 FROM country y  
WHERE y.continent = x.continent  
AND y.name != x.name);
```



OUTPUT DATA:

VIVA QUESTIONS:

EXPERIMENT: 3

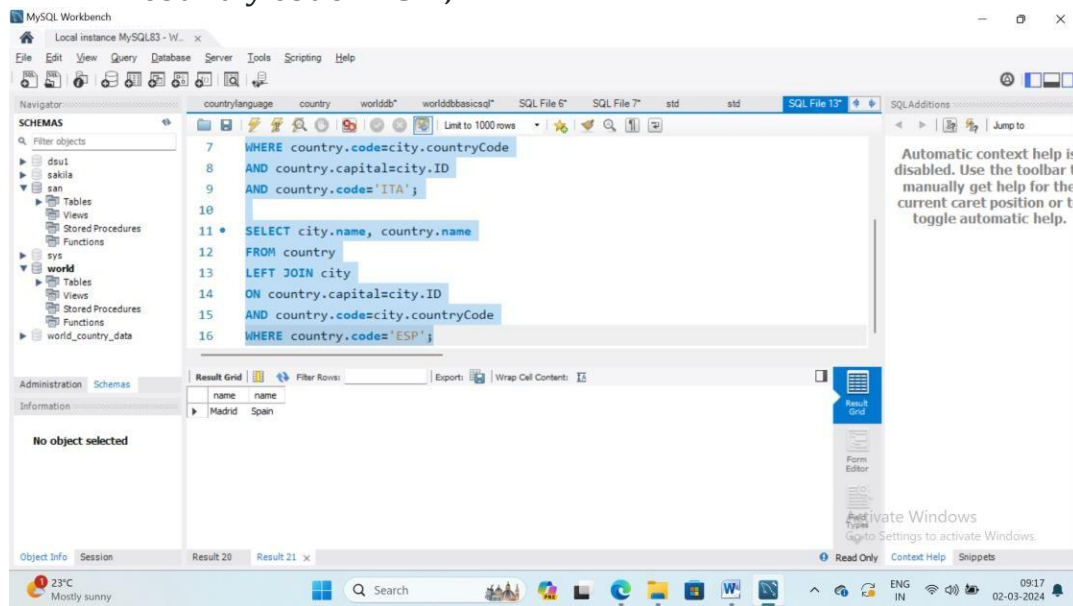
AIM: Write Sql code to join tables and retrieve the combined information from tables.

PROGRAM:

- a) Using LEFT JOIN, ON, what is the capital of Spain (ESP)? (Madrid)

```
SELECT city.name
FROM city, country
WHERE country.code=city.countryCode
AND country.capital=city.ID
AND country.code='ITA';
```

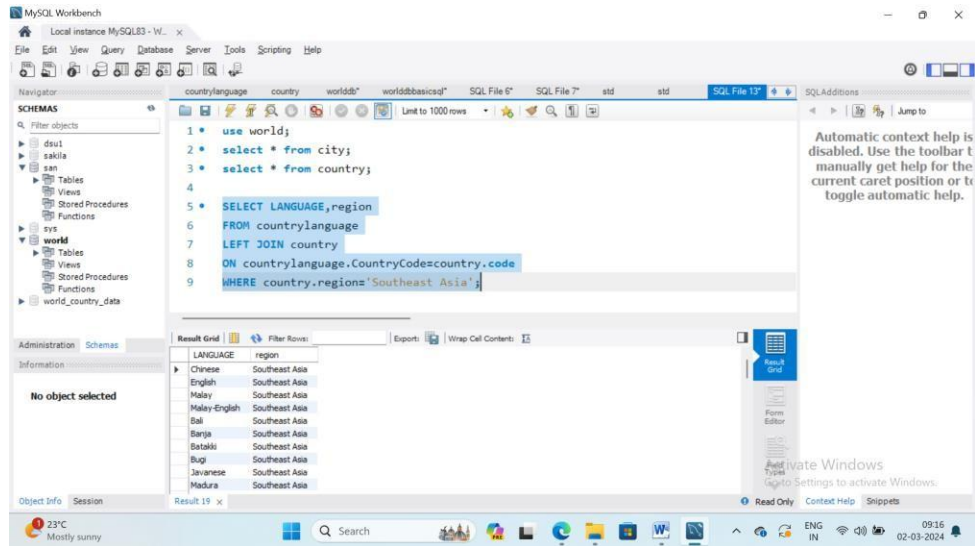
```
SELECT city.name, country.name
FROM country
LEFT JOIN city
ON country.capital=city.ID
AND country.code=city.countryCode
WHERE country.code='ESP';
```



- b) Using LEFT JOIN, ON, list all the languages spoken in the 'Southeast Asia' region (65)

```
SELECT LANGUAGE,region
FROM countrylanguage
LEFT JOIN country
ON countrylanguage.CountryCode=country.code
```

WHERE country.region='Southeast Asia';



Extra code for practice:

```
SELECT column_name(s)
FROM table1
INNER JOIN table2
ON table1.column_name = table2.column_name;
```

```
SELECT column_name(s)
FROM table1
RIGHT JOIN table2
ON table1.column_name = table2.column_name;
```

```
SELECT column_name(s)
FROM table1
LEFT JOIN table2
ON table1.column_name = table2.column_name;
```

```
SELECT column_name(s)
FROM table1
FULL OUTER JOIN table2
ON table1.column_name = table2.column_name
WHERE condition;
```

VIVA QUESTIONS:

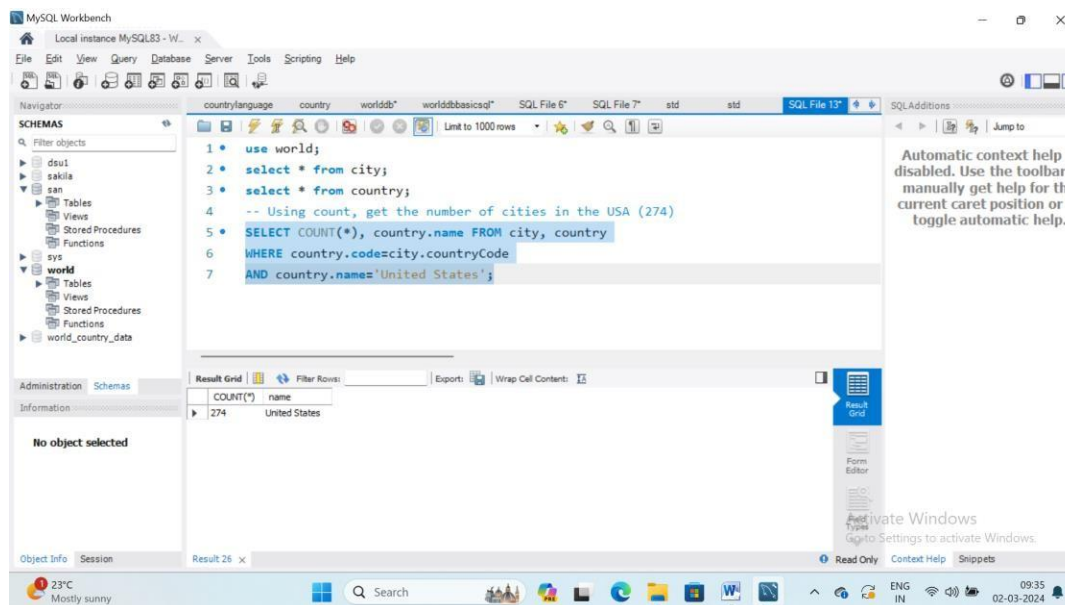
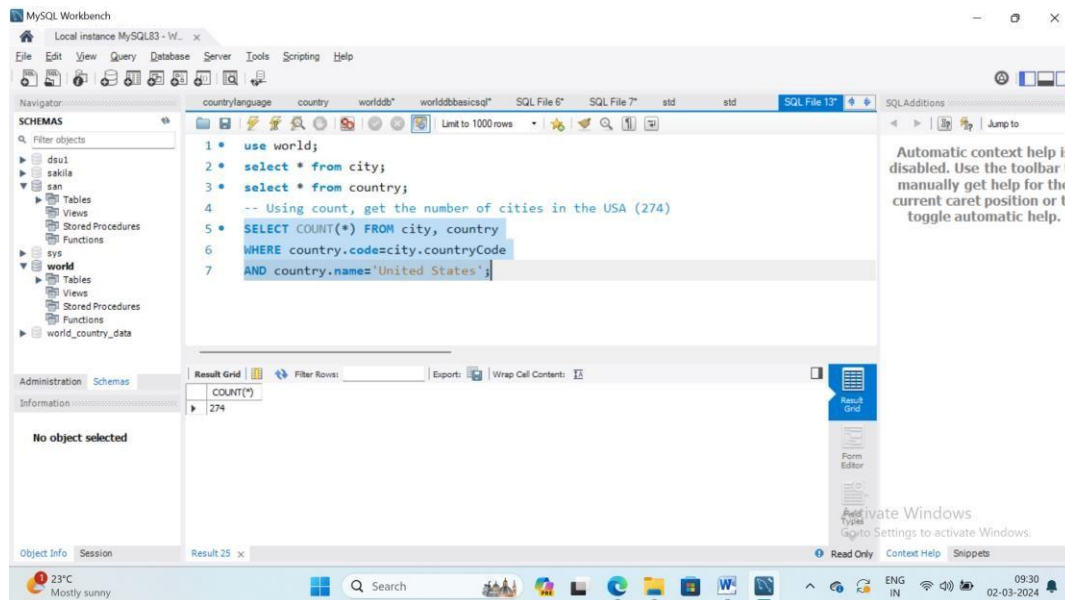
EXPERIMENT: 4

AIM: Execute the Aggregate functions count, sum, avg, min, max on a suitable database. Make use of built in functions according to the need of the database

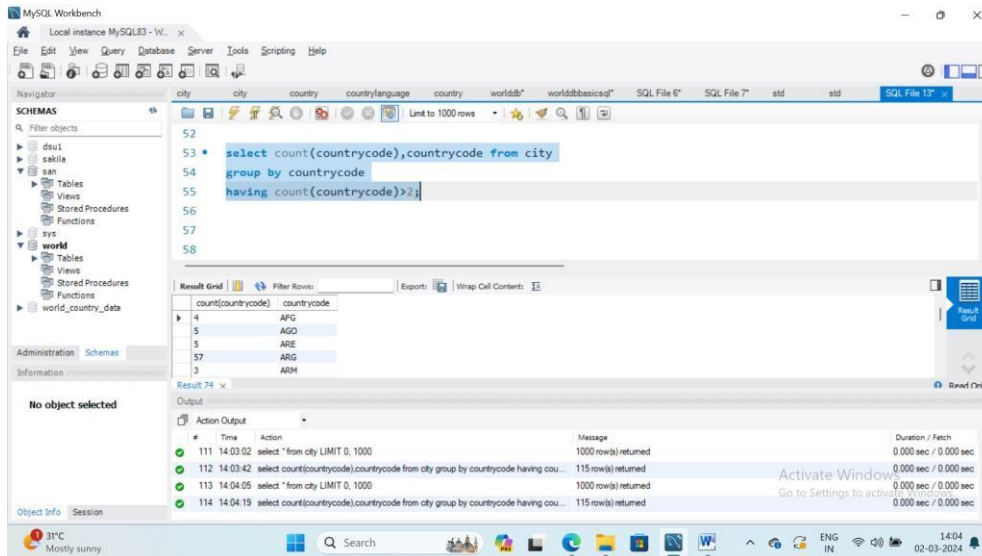
PROGRAM:

a) Using count, get the number of cities in the USA (274)

```
SELECT COUNT(*) FROM city, country
WHERE country.code=city.countryCode
AND country.name='United States';
```

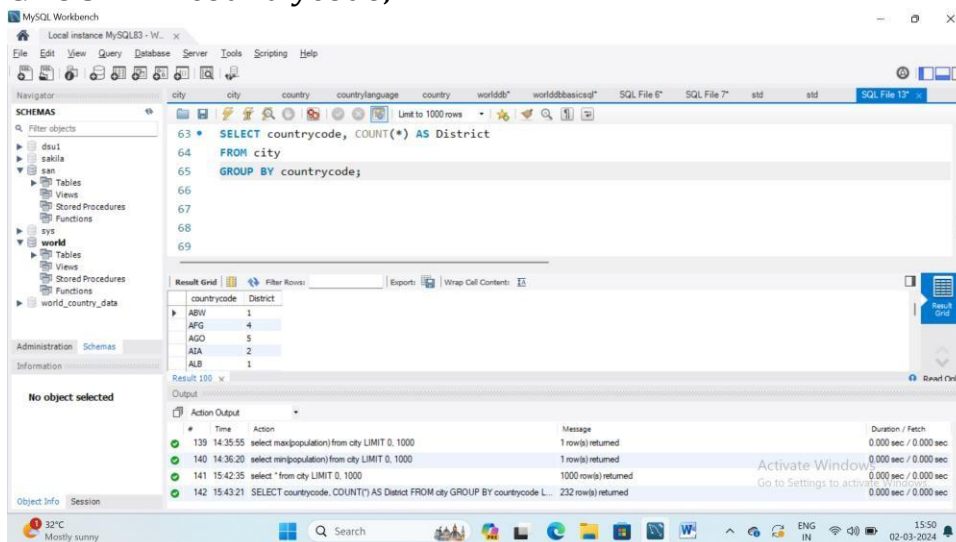


- b) Get count of coutrycode where countrycode > 100
select count(countrycode),countrycode from city
group by countrycode
having count(countrycode)>100;



- c) Find total districts in each country.

SELECT countrycode, country,COUNT(*) AS District
FROM city
GROUP BY countrycode;



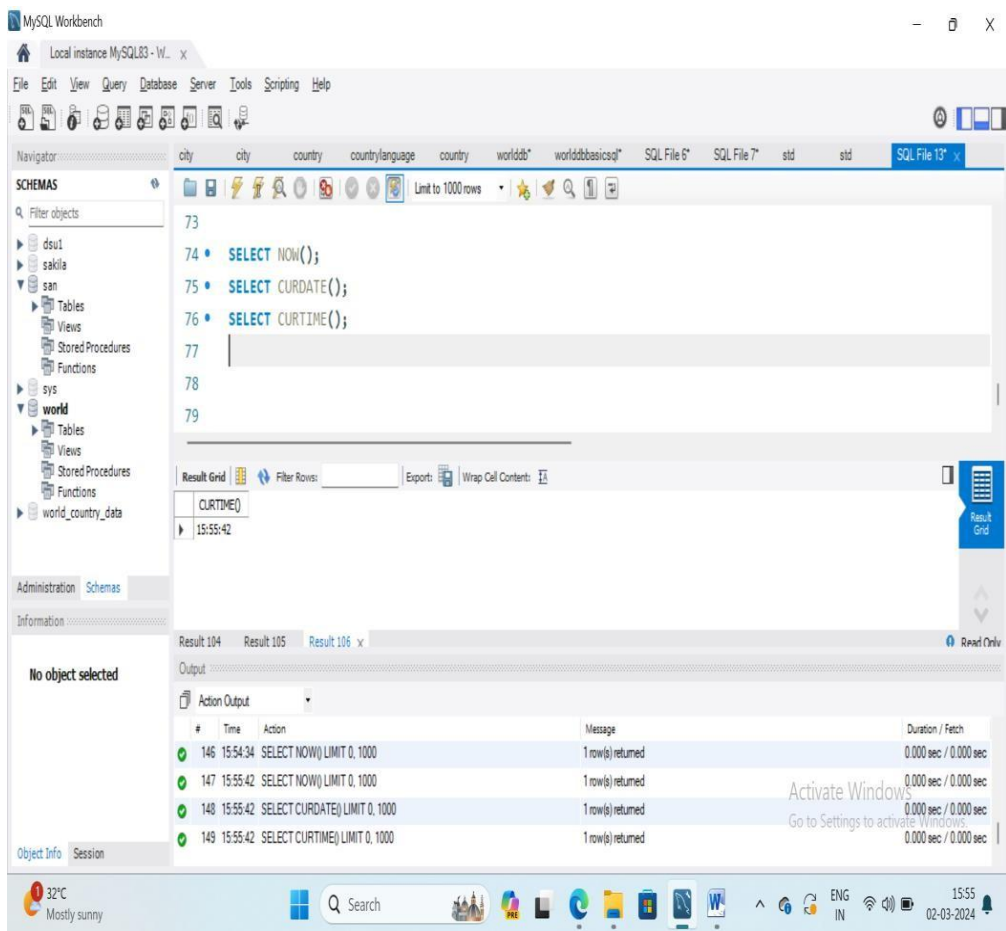
VIVA QUESTIONS:

EXPERIMENT: 5

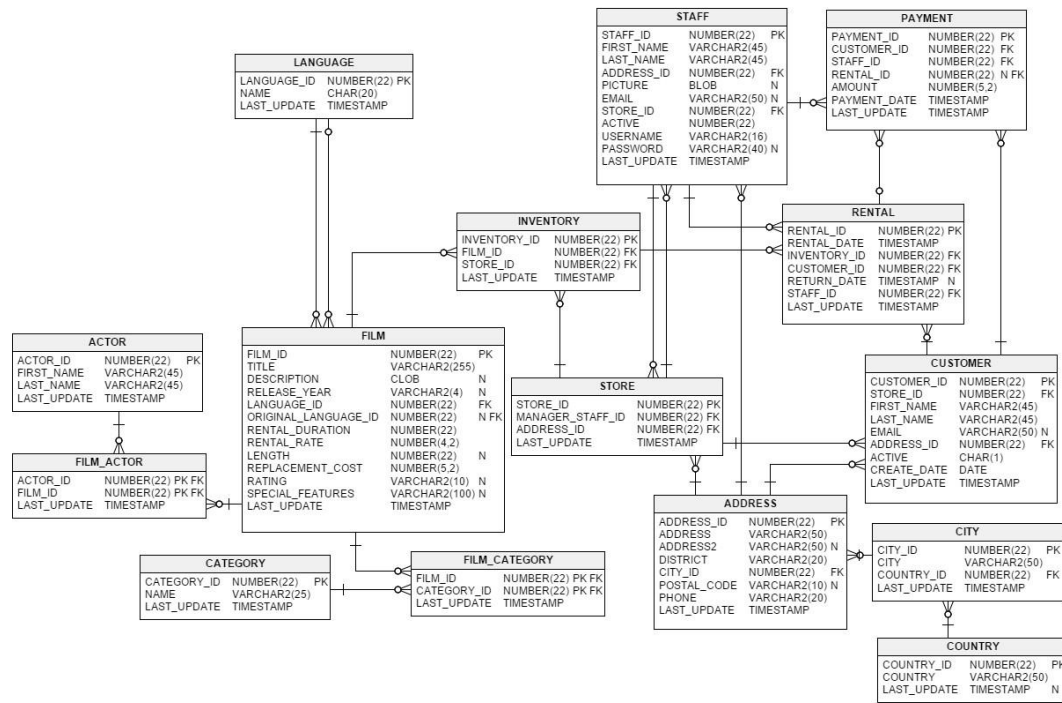
AIM: Retrieve the data from the database based on time and date functions like now(), date(), day(), time() etc., Use of group by and having clauses.

PROGRAM:

- a) `SELECT NOW();`
`SELECT CURDATE();`
`SELECT CURTIME();`

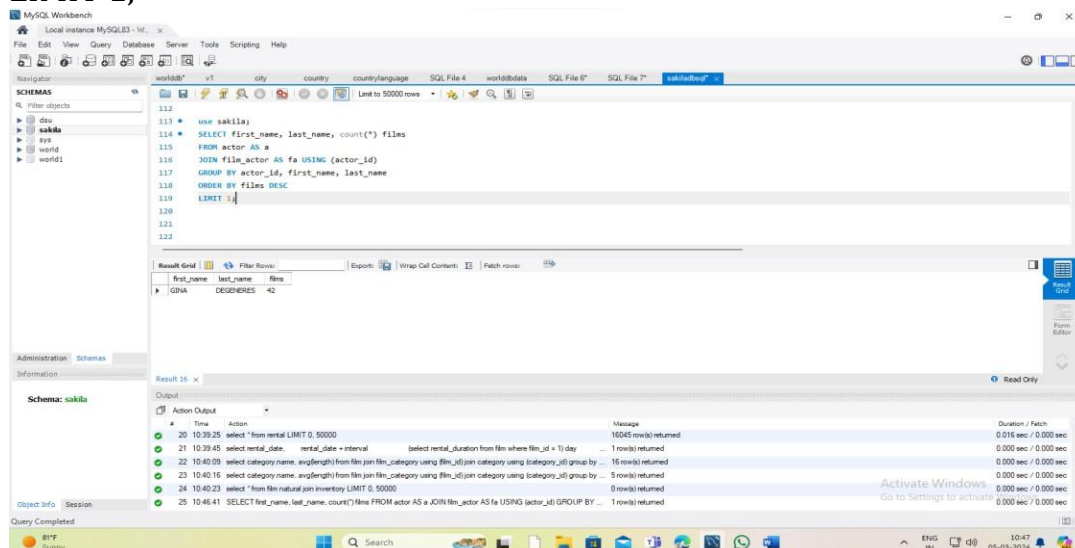


Sakila database schema or ERD



b) Actor with most films (ignoring ties) from sakila database

SELECT first_name, last_name, count(*) films
 FROM actor AS a
 JOIN film_actor AS fa USING (actor_id)
 GROUP BY actor_id, first_name, last_name
 ORDER BY films DESC
 LIMIT 1;



c) Cumulative revenue of all stores

SELECT payment_date, amount, sum(amount) OVER (ORDER BY payment_date) FROM (SELECT CAST(payment_date AS DATE) AS payment_date, SUM(amount) AS amount FROM payment GROUP BY CAST(payment_date AS DATE)) ORDER BY payment_date;

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
131
132 use sakila;
133 SELECT payment_date, amount, sum(amount) OVER (ORDER BY payment_date)
134 FROM (SELECT CAST(payment_date AS DATE) AS payment_date, SUM(amount)
135 AS amount
136 FROM payment
137 GROUP BY CAST(payment_date AS DATE)) p
138 ORDER BY payment_date;
```

The Results tab displays the output of the query:

payment_date	amount	sum(amount) OVER (ORDER BY payment_date)
2005-05-24	29.92	29.92
2005-05-25	573.63	603.55
2005-05-26	754.26	1357.81
2005-05-27	684.24	2042.05
2005-05-28	804.04	2846.09
2005-05-29	646.46	3494.65
2005-05-30	626.42	4123.07
2005-05-31	700.37	4823.44

The bottom of the screenshot shows the 'Action Output' tab with a list of executed queries and their execution times.

VIVA QUESTIONS:

EXPERIMENT: 6

AIM: Write and execute database trigger. Consider row level and statement level triggers.

PROGRAM:

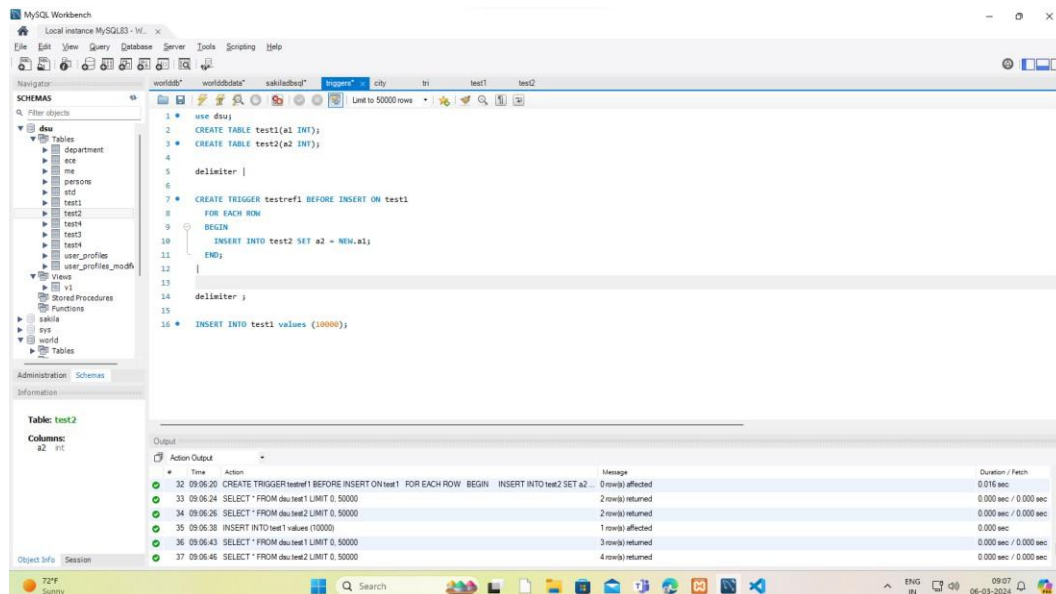
```
CREATE TABLE test1(a1 INT);  
CREATE TABLE test2(a2 INT);
```

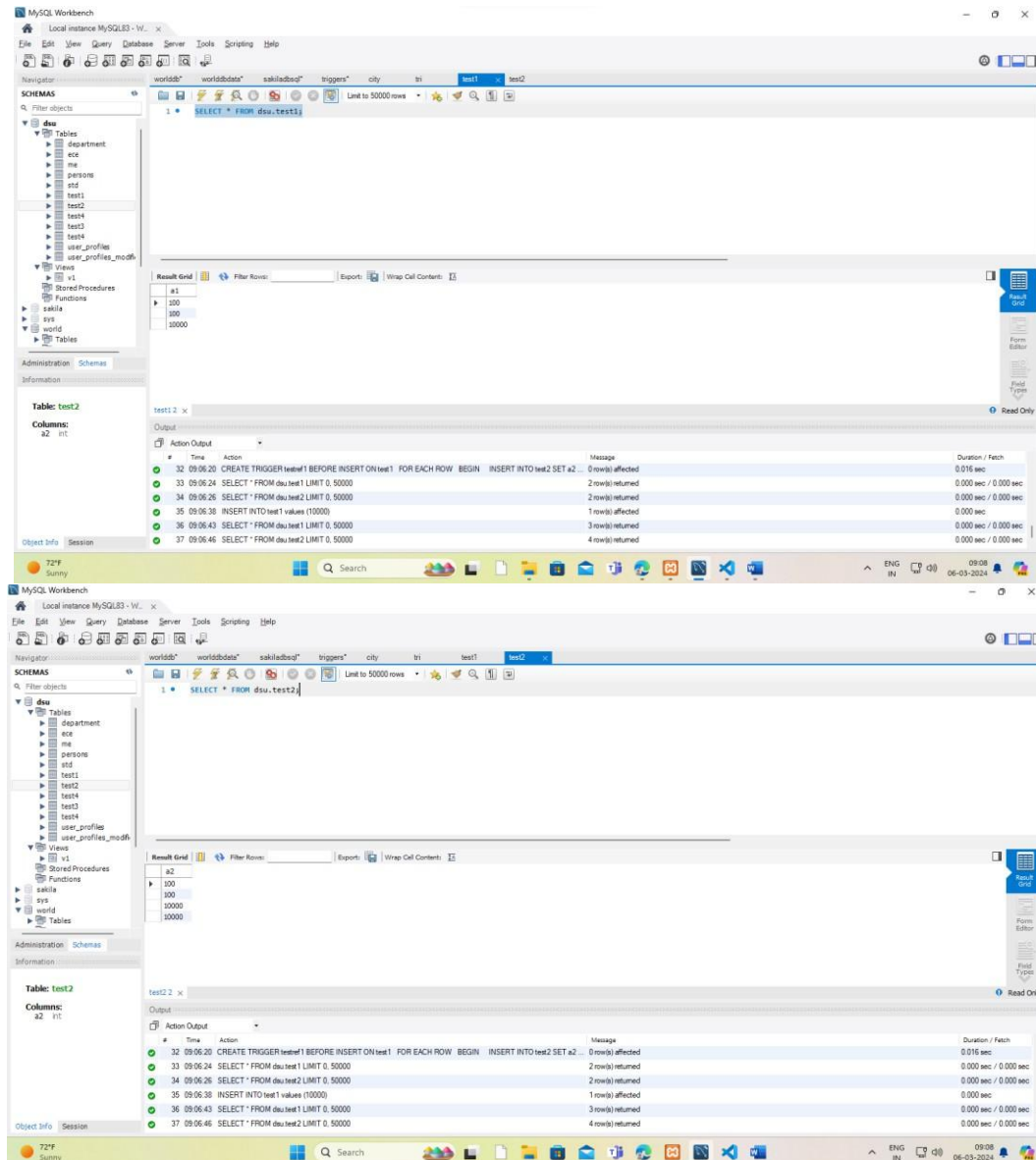
```
delimiter |
```

```
CREATE TRIGGER testref1 BEFORE INSERT ON test1  
FOR EACH ROW  
BEGIN  
INSERT INTO test2 SET a2 = NEW.a1;  
END;  
|
```

```
delimiter ;
```

```
INSERT INTO test1 values (10000);
```





```
CREATE TABLE test1(a1 INT);
CREATE TABLE test2(a2 INT);
CREATE TABLE test3(a3 INT NOT NULL AUTO_INCREMENT PRIMARY KEY);
CREATE TABLE test4(
a4 INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
b4 INT DEFAULT 0
);
```

delimiter |

```
CREATE TRIGGER testref BEFORE INSERT ON test1
FOR EACH ROW
BEGIN
```

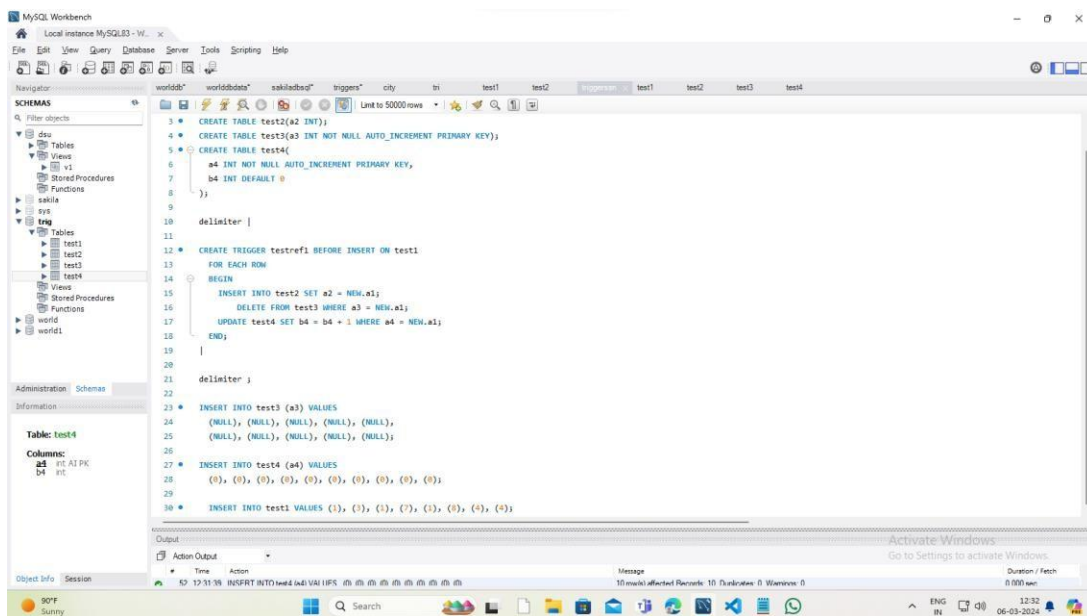
```
INSERT INTO test2 SET a2 = NEW.a1;  
DELETE FROM test3 WHERE a3 = NEW.a1;  
UPDATE test4 SET b4 = b4 + 1 WHERE a4 = NEW.a1;  
END;
```

|

delimiter ;

```
INSERT INTO test3 (a3) VALUES  
(NULL), (NULL), (NULL), (NULL), (NULL),  
(NULL), (NULL), (NULL), (NULL), (NULL);
```

```
INSERT INTO test4 (a4) VALUES  
(0), (0), (0), (0), (0), (0), (0), (0), (0), (0);
```



MySQL Workbench

Local instance MySQL83 - W...

File Edit View Query Database Server Tools Scripting Help

Navigator

worlds* worldbdata* sakila* triggers* city tri test1 test2 triggeran test1 test2 test3 test4

SCHEMAS

Filter objects

dtu

Tables

Views

v1

Stored Procedures

Functions

sakila

sys

trig

Tables

test1

test2

test3

test4

Views

Stored Procedures

Functions

world

world1

Administration Schemas

Information

Table: test4

Columns:

id1 int AI PK

id2 int

Result Grid

Filter Rows:

Export:

Wrap Cell Contents: IS

test1 2 x

Output

Action Output

Message

10 rows affected Records: 10 Duration: 0 Warnings: 0

Duration / Fetch

0.000 sec

Object Info Session

12:33 06-03-2024

MySQL Workbench

Local instance MySQL83 - W...

File Edit View Query Database Server Tools Scripting Help

Navigator

worlds* worldbdata* sakila* triggers* city tri test1 test2 triggeran test1 test2 test3 test4

SCHEMAS

Filter objects

dtu

Tables

Views

v1

Stored Procedures

Functions

sakila

sys

trig

Tables

test1

test2

test3

test4

Views

Stored Procedures

Functions

world

world1

Administration Schemas

Information

Table: test4

Columns:

id1 int AI PK

id2 int

Result Grid

Filter Rows:

Export:

Wrap Cell Contents: IS

test1 2 x

Output

Action Output

Message

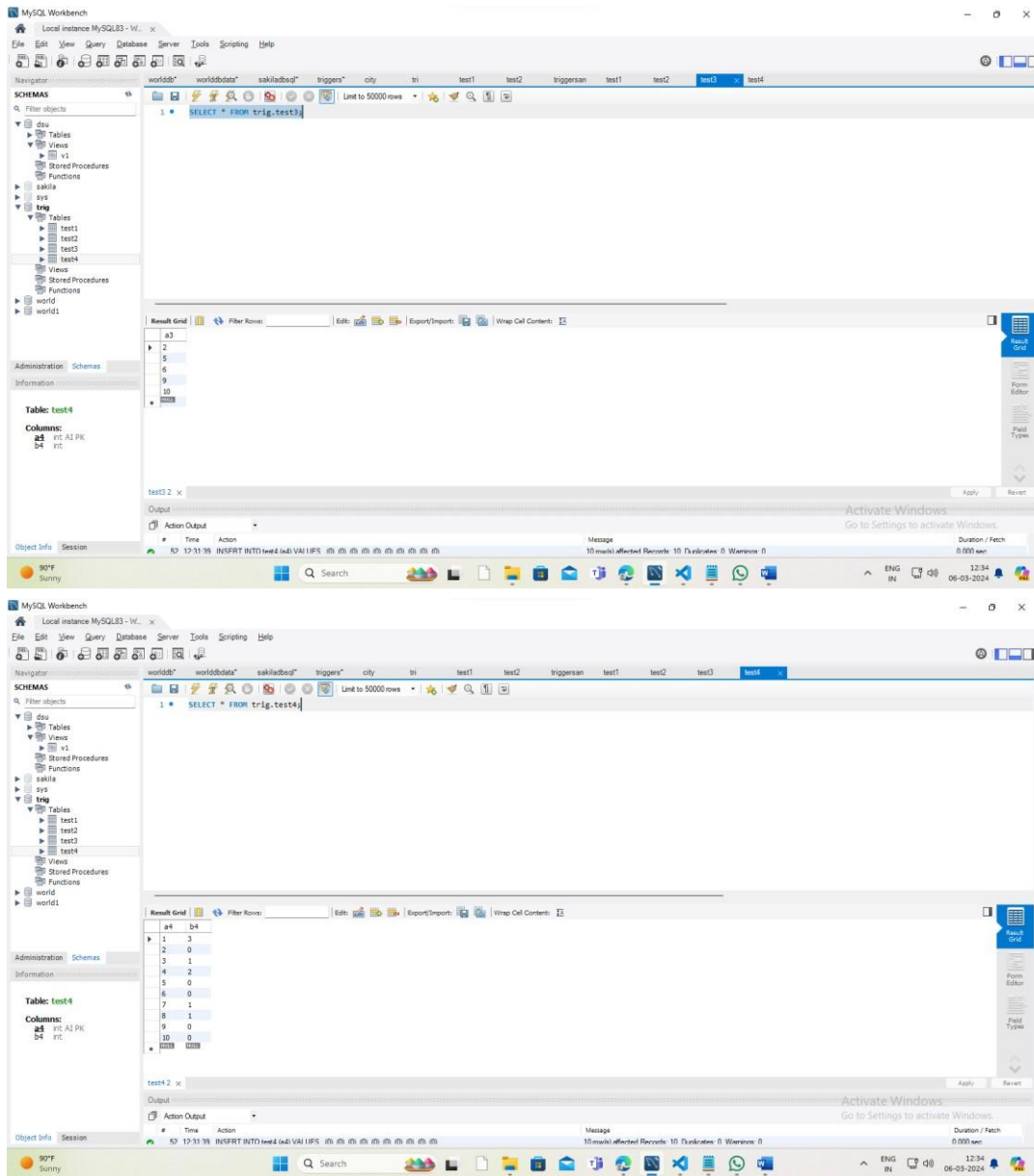
10 rows affected Records: 10 Duration: 0 Warnings: 0

Duration / Fetch

0.000 sec

Object Info Session

12:33 06-03-2024



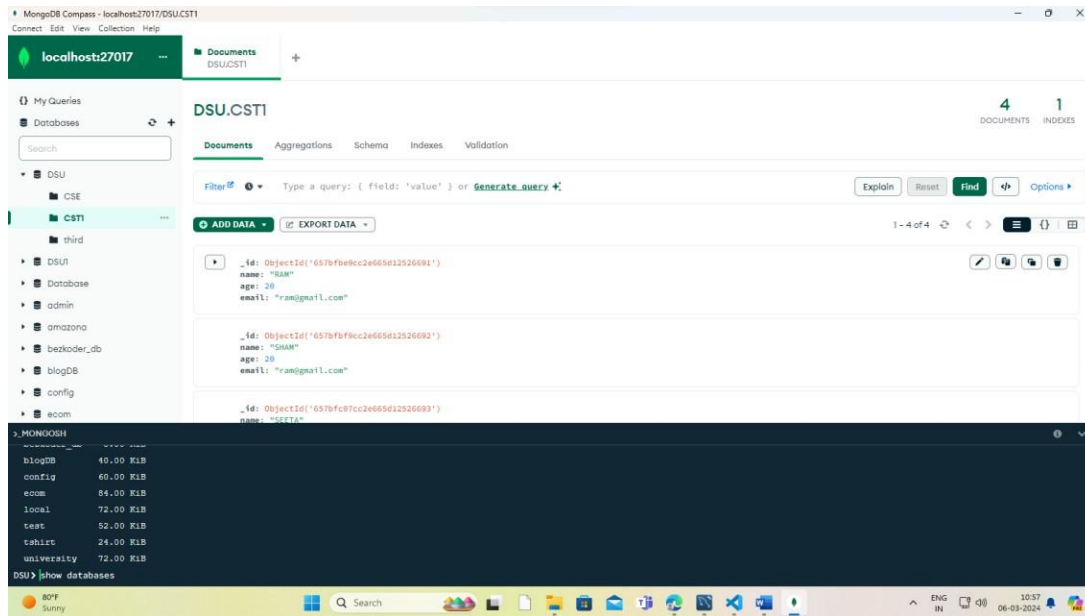
VIVA QUESTIONS:

EXPERIMENT: 7

AIM: Write and execute program to perform operations on MongoDB Database.

PROGRAM:

a) Show databases



a) Use databasename

b) insertone()

c) insertmany()

d) Findone()

e) Findmany()

mongodb download and install

mongodb community server download and install

```
// const mongo1 = require('mongodb').MongoClient;  
// const url1 = 'mongodb://localhost:27017';  
// const client1 = new mongo1(url1);  
// const dbname = 'e-comm';
```

```
// function f1() {  
//   let conn = client1.connect(); // connect to mongo  
//   let dbt = conn.db(dbname); // connect to table of db
```

```

//    return dbt.collection('product')
// }

// module.exports=f1;
// const MongoClient = require('mongodb').MongoClient
// const url1 = 'mongodb://localhost:27017';
// const client = new MongoClient(url1);
// const dbname = 'e-comm';

// async function f1() {
//   let result = await client.connect(); // connect mongo db
//   let db = result.db(dbname)          // which data base
//   let collection = db.collection('product') // which table
//   let response = await collection.find({}).toArray()
//   console.log(response)
// }
// f1()

//=====
// download and install mongodb in local system
// install package in node ... using
// npm install mongodb
// npm install express // additional framework
// npm install nodemon // additional package for application running
// import using require function with MongoClient
// const MongoClient = require('./mongodb').MongoClient
// To MongoClient object pass mongodb URL so that
// node will understand which port mongodb connected
// const client = new MongoClient('mongodb://localhost:27017')
// client.connect to connect to mongodb and node
// const result = client.connect()
// then const xx = result.db(dbname) to connect specific database
// const yy = xx.collection(tablename) to connect specific table
// then yy.find() to fetch all data from table
// async and await are used to handle (promise) delay
//=====

////////// data fetch from table
const MC = require('mongodb').MongoClient
const url2 = 'mongodb://localhost:27017'
const res = new MC(url2)
const dbname2 = 'e-comm'

async function f2() {
  let xx = await res.connect()
  let yy = xx.db(dbname2)

```

```
let zz = yy.collection('product')
let r1 = await zz.find({}).toArray()
console.log(r1)
}
```

```
f2()
```

```
////////// data insertion in to table
```

```
async function f3() {
let xx = await res.connect()
let yy = xx.db(dbname2)
let zz = yy.collection('product')
let r1 = await zz.insertOne({
"name": "S!!!!",
"brand": "S1",
"price": 250,
"cat": "S11111"
})
if((await r1).acknowledged) {
console.log('data inserted')}
else {console.log(' not inserted')}
}
```

```
f3()
```

```
//////// data updation in to table //////////////////////////////////////
```

```
async function f4() {
let xx = await res.connect()
let yy = xx.db(dbname2)
let zz = yy.collection('product')
let r1 = await zz.updateOne(
{name:'s-40'},
{$set:{name:'s111111-40'}} )
)
console.warn(r1)
}
f4()
```

```
////////// delete the data from mongodb
```

```
async function f5() {
let xx = await res.connect()
let yy = xx.db(dbname2)
let zz = yy.collection('product')
let r1 = await zz.deleteOne({name:'s-40'})
console.warn(r1)
}
```

f50
//
//

- 1) Show dbs;
- 2) Create a database DSU than
- 3) Use DSU
- 4) Create collection(table)
- 5) DSU> db.cse.insertOne({ name:'Chirag111', age:20, sal: 50000})
- 6) db.cse.find()
- 7) DSU> db.cse.insertOne({ name:'Chirag111', age:20, sal: 50000})
- 8) DSU> db.cse.insertOne({ name:'Chirag222', age:20, sal: 50000})
- 9) DSU>db.cse.find()
- 10) DSU>db.cse.findOne({name:'Chirag'})
- 11) DSU>db.cse.findOne({name:'Chirag111'})
- 12) Use databasename
- 13) Show collections
- 14) Db.createCollection ('name of collection')
- 15) Db.collectionname.find();
- 16) Db.collectionname.findOne({name:'xyz'})
- 17) Db.collectionname.insertOne({ name:'ddd', age:20, sal: 50000})

VIVA QUESTIONS:

Experiment 8

Aim: Write and execute program to perform CRUD operations.

Program:

Create record, read record, update record and delete the record

a) Create operation

```
insertOne({  
  fName: 'RAM',  
  lName: 'AYODHYA',  
  email: 'ram@gmail.com',  
  hobbies: ['Photography', 'Travelling']  
});
```

```
insertMany([ {  
  fName: 'Adi',  
  lName: 'Yogi',  
  email: 'yogi@gmail.com',  
  hobbies: ['Drawing', 'Music']  
}, {  
  fName: 'Modi',  
  lName: 'PM',  
  email: 'modi@gmail.com',  
  hobbies: ['Drawing', 'Classical Dance'] } ]]);
```

b) Read operation

```
collection.find()
```

Output Data

VIVA Questions

Mini Project:

- For any problem selected, write the ER Diagram, apply ER-mapping rules, normalize the relations, and follow the application development process.
- Make sure that the application should have five or more tables, at least one trigger and one stored procedure, using suitable frontend tool.
- Indicative areas include: health care, education, industry, transport, supply chain etc.

OPEN ENDED EXPERIMENTS

1. Implement the cursor function to process the result set in a row by row manner.
2. Implement the pivot functions to perform cross tabulation.
3. Implement Composite indexes that are created by using multiple columns to uniquely identify the data points.

Extra sql code:

SQL Homework

USE sakila;

1a. Display the first and last names of all actors from the table `actor`.

```
SELECT first_name, last_name  
FROM actor;
```

1b. Display the first and last name of each actor in a single column in upper case letters. Name the column `Actor Name`.

```
SELECT CONCAT (first_name, ' ', last_name) as 'Actor Name'  
FROM actor;
```

2a. You need to find the ID number, first name, and last name of an actor, of whom you know only the first name, "Joe."

What is one query you would use to obtain this information?

```
SELECT actor_id, first_name, last_name  
FROM actor  
WHERE first_name = "Joe";
```

2b. Find all actors whose last name contain the letters `GEN`:

```
SELECT first_name, last_name  
FROM actor  
WHERE last_name LIKE "%GEN%";
```

2c. Find all actors whose last names contain the letters `LI`.

This time, order the rows by last name and first name, in that order:

```
SELECT last_name, first_name  
FROM actor  
WHERE last_name LIKE "%LI%"  
ORDER BY last_name;
```

2d. Using `IN`, display the `country\_id` and `country` columns of the following countries: Afghanistan, Bangladesh, and China:

```
SELECT country_id, country  
FROM country  
WHERE country IN ('Afghanistan', 'Bangladesh', 'China');
```

3a. Add a `middle\_name` column to the table `actor`.

Position it between `first\_name` and `last\_name`. Hint: you will need to specify the data # type.

To get the data type of the columns

```
DESCRIBE actor;
```

```
ALTER TABLE actor
```

```
ADD middle_name varchar(45) NOT NULL AFTER first_name;
```

```
SELECT * FROM actor;
```

3b. You realize that some of these actors have tremendously long last names.

Change the data type of the `middle\_name` column to `blobs`.

```
ALTER TABLE actor
```

```
MODIFY middle_name BLOB NOT NULL;
```

To get the data type of the columns

```
DESCRIBE actor;
```

3c. Now delete the `middle\_name` column.

```
ALTER TABLE actor
```

```
DROP middle_name;
```

```
DESCRIBE actor;
```

4a. List the last names of actors, as well as how many actors have that last name.

```
SELECT last_name, COUNT(*) AS 'Number of actors with last name'
```

```
FROM actor
```

```
GROUP BY last_name;
```

4b. List last names of actors and the number of actors who have that last name, but only for names that are shared by at least two actors.

```
SELECT last_name, COUNT(*) AS COUNT
```

```
FROM actor
```

```
GROUP BY last_name
```

```
HAVING COUNT >= 2;
```

4c. Oh, no! The actor `HARPO WILLIAMS` was accidentally entered in the `actor` table as # `GROUCHO WILLIAMS`,

the name of Harpo's second cousin's husband's yoga teacher. Write a query to fix the record,

```
UPDATE actor SET first_name = REPLACE(first_name, 'GROUCHO','HARPO');
```

There should be no actor named 'GROUCHO WILLIAMS' after the above

query is run.

To check run the following query.

```
SELECT * FROM actor
WHERE first_name = "GROUCHO";
```

Check to see the change

```
SELECT * FROM actor
WHERE first_name = "HARPO";
```

4d. Perhaps we were too hasty in changing `GROUCHO` to `HARPO`.

It turns out that `GROUCHO` was the correct name after all!

In a single query, if the first name of the actor is currently `HARPO`, change it to #`GROUCHO`.

Otherwise, change the first name to `MUCHO GROUCHO`, as that is exactly what the #actor will be with the grievous error.

BE CAREFUL NOT TO CHANGE THE FIRST NAME OF EVERY ACTOR TO #`MUCHO GROUCHO`,

HOWEVER! (Hint: update the record using a unique identifier.)

```
UPDATE actor SET first_name = REPLACE(first_name, 'HARPO', 'GROUCHO');
SELECT * FROM actor
WHERE first_name = "GROUCHO";
```

##UPDATE actor SET first_name = REPLACE(first_name, 'MUCHO MUCHO MUCHO #GROUCHO', 'GROUCHO')

##WHERE last_name = "WILLIAMS";

##UPDATE actor SET first_name = REPLACE(first_name, 'GROUCHO', 'HARPO')

##WHERE last_name = "WILLIAMS";

Check to see the change

```
SELECT * FROM actor
WHERE last_name = "WILLIAMS";
```

5a. You cannot locate the schema of the `address` table. Which query would you use to re-create it?

```
SHOW CREATE TABLE address;
```

6a. Use `JOIN` to display the first and last names, as well as the address, of each staff member. Use the tables `staff` and `address`

```
SELECT s.first_name, s.last_name, a.address
FROM staff AS s
LEFT JOIN address AS a
ON (s.address_id = a.address_id);
```

6b. Use `JOIN` to display the total amount rung up by each staff member in August of 2005. Use tables `staff` and `payment`.

This gets the total amount for August 2005.

```
SELECT CONCAT("$",FORMAT(SUM(amount),2)) AS Total FROM payment
WHERE payment_date LIKE '2005-08%';
```

This gets the total from each staff member. The amounts should add up to the total from the above query.

```
SELECT s.first_name, s.last_name, p.staff_id,
CONCAT("$",FORMAT(SUM(amount),2)) AS Total
FROM staff AS s
JOIN payment AS p
ON (s.staff_id = p.staff_id) AND p.payment_date LIKE '2005-08%'
GROUP BY p.staff_id;
```

6c. List each film and the number of actors who are listed for that film. Use tables `film\_actor` and `film`. Use inner join.

```
SELECT f.title, COUNT(fa.actor_id) AS 'Number of Actors'
FROM film AS f
INNER JOIN film_actor AS fa
ON (f.film_id = fa.film_id)
GROUP BY f.film_id;
```

6d. How many copies of the film `Hunchback Impossible` exist in the inventory system?

```
SELECT f.title, (
SELECT COUNT(*) FROM inventory AS i
WHERE f.film_id = i.film_id
) AS 'Number of Copies'
FROM film AS f
WHERE f.title = "Hunchback Impossible";
```

6e. Using the tables `payment` and `customer` and the `JOIN` command,

list the total paid by each customer.

List the customers alphabetically by last name:

The total paid by each customer from the Payment table.

```
SELECT customer_id, SUM(amount) AS 'Total Paid'
FROM payment
GROUP BY customer_id;
```

Displaying all the customers alphabetically by last name.

```
SELECT last_name, first_name, customer_id
FROM customer
ORDER BY last_name ASC;
```

Combining the two with 'JOIN'

```
SELECT c.first_name, c.last_name, CONCAT("$",FORMAT(SUM(p.amount),2)) AS
'Total Amount Paid'
FROM payment AS p
JOIN customer AS c
ON (p.customer_id = c.customer_id)
GROUP BY p.customer_id
ORDER BY c.last_name ASC;
```

7a. The music of Queen and Kris Kristofferson have seen an unlikely resurgence.

As an unintended consequence, films starting with the letters `K` and `Q` have also # # # # soared in popularity.

Use subqueries to display the titles of movies starting with the letters `K` and `Q` whose # language is English.

```
SELECT f.title
FROM film AS f
WHERE f.language_id IN (
SELECT l.language_id
FROM language AS l
WHERE f.title LIKE 'K%' OR f.title LIKE 'Q%' AND l.name = "English"
);
```

The movies that start with a "Q" or a "K" are all in English anyway.

```
SELECT title, language_id
FROM film
WHERE title LIKE 'K%' OR title LIKE 'Q%';
```

7b. Use subqueries to display all actors who appear in the film `Alone Trip`.

```
SELECT first_name, last_name
FROM actor
WHERE actor_id IN (
SELECT actor_id
FROM film_actor
WHERE film_id IN (
SELECT film_id
FROM film
WHERE title = "Alone Trip")
);
```

**## 7c. You want to run an email marketing campaign in Canada,
for which you will need the names and email addresses of all Canadian
customers. Use joins to retrieve this information**

```
SELECT c.first_name, c.last_name, c.email, country.country
FROM address AS a
JOIN city
ON (city.city_id = a.city_id)
JOIN customer AS c
ON (c.address_id = a.address_id)
JOIN country
ON (country.country_id = city.country_id)
WHERE country.country = 'Canada';
```

**## 7d. Sales have been lagging among young families, and you wish to target
all family movies for a promotion.
Identify all movies categorized as family films.**

```
SELECT f.title, c.name AS 'Type of Film'
FROM category AS c
JOIN film_category AS fc
ON (fc.category_id = c.category_id)
JOIN film AS f
ON (f.film_id = fc.film_id)
WHERE c.name = "Family";
```

7e. Display the most frequently rented movies in descending order.

```
SELECT f.title, COUNT(f.title) AS 'Number_Rented'
FROM film AS f
JOIN inventory AS i
ON (i.film_id = f.film_id)
JOIN rental AS r
ON (i.inventory_id = r.inventory_id)
GROUP BY f.title
ORDER BY COUNT(f.title) DESC;
```

7f. Write a query to display how much business, in dollars, each store brought in.

```
SELECT s.store_id, CONCAT("$",FORMAT(SUM(p.amount),2)) AS 'Money Made'
FROM store AS s
JOIN inventory AS i
ON(i.store_id = s.store_id)
JOIN rental AS r
ON (r.inventory_id = i.inventory_id)
JOIN payment AS p
ON(p.rental_id = r.rental_id)
GROUP BY s.store_id;
```

7g. Write a query to display for each store its store ID, city, and country.

```
SELECT s.store_id, city.city, country
FROM store AS s
LEFT JOIN address AS a
ON(a.address_id = s.address_id)
INNER JOIN city
ON (city.city_id = a.city_id)
INNER JOIN country AS c
ON(c.country_id = city.country_id)
GROUP BY s.store_id;
```

7h. List the top five genres in gross revenue in descending order.

(Hint**: you may need to use the following tables: category, film_category, inventory, payment, and rental.)**

```
SELECT cat.name AS 'Film Genre', CONCAT("$",FORMAT(SUM(p.amount),2)) AS
'Gross Revenue'
FROM category AS cat
JOIN film_category AS fc
ON(fc.category_id = cat.category_id)
JOIN inventory AS i
ON (i.film_id = fc.film_id)
JOIN rental AS r
ON(r.inventory_id = i.inventory_id)
JOIN payment AS p
ON(p.rental_id = r.rental_id)
GROUP BY cat.name
ORDER BY SUM(p.amount) DESC
LIMIT 5;
```

8a. In your new role as an executive, you would like to have an easy way of viewing the Top five genres by gross revenue.

Use the solution from the problem above to create a view.

If you haven't solved 7h, you can substitute another query to create a view.

```
CREATE VIEW top_five_genres AS
SELECT cat.name AS 'Film Genre', CONCAT("$",FORMAT(SUM(p.amount),2)) AS
'Gross Revenue'
FROM category AS cat
JOIN film_category AS fc
ON(fc.category_id = cat.category_id)
JOIN inventory AS i
ON (i.film_id = fc.film_id)
JOIN rental AS r
ON(r.inventory_id = i.inventory_id)
JOIN payment AS p
ON(p.rental_id = r.rental_id)
GROUP BY cat.name
ORDER BY SUM(p.amount) DESC
LIMIT 5;
```

8b. How would you display the view that you created in 8a?

```
SELECT * FROM top_five_genres;
```

8c. You find that you no longer need the view `top\_five\_genres`. Write a query to delete it.

```
DROP VIEW top_five_genres;
```

Extra code on sakila db.....

-- which actors have the first name 'scarlett'

```
select first_name, last_name
```

```
from actor
```

```
where first_name = 'scarlett';
```

-- which actors have the last name 'johansson'

```
select first_name, last_name
```

```
from actor
```

```
where last_name = 'johansson';
```

-- how many distinct actors last name are there

```
select distinct last_name, count(last_name)
```

```
from actor
```

```
group by last_name;
```

-- which last names are not repeated

```
select distinct last_name,count(last_name) as 'not repeated'
```

```
from actor
```

```
group by last_name
having count(last_name) = '1'
order by count(last_name);
```

```
-- which last names appear more than once
select last_name, count(last_name) as 'repeated'
from actor
group by last_name
having count(last_name) > '1'
order by count(last_name);
```

```
-- which actor has appeared in most films
```

```
select a.first_name, a.last_name, count(fa.actor_id) as 'film count'
from actor as a
inner join film_actor as fa on a.actor_id = fa.actor_id
group by a.actor_id
order by count(a.actor_id) desc;
```

```
-- is academy dinosaur available for rent in store 1
```

```
select *
from film as f
inner join inventory as i on f.film_id = i.inventory_id
inner join rental as r on r.inventory_id = i.inventory_id
where title = 'academy dinosaur' and store_id = '1';
-- OR
select title, film_id, inventory_id, store_id
from inventory join film using (film_id) join store using (store_id)
where title = 'academy dinosaur' and store_id = '1';
```

```
-- what is the average running time of all movies in the sakila database?
```

```
select round(avg(length))
from film;
```

```
-- what is the average running time of film by category
```

```
select name, round(avg(length))
from film as f
inner join film_category as fc on f.film_id = fc.film_id
inner join category as c on c.category_id = fc.category_id
group by c.name
order by round(avg(length)) desc;
```

```
-- display the first name and last name of all actors from the table actor
```

```
select first_name, last_name
```

```
from actor;
```

```
-- display the first name and last name of all actors in a single column in upper  
case letters. name the column actor name.
```

```
select upper (concat(first_name,' ',last_name)) as 'Actor name'  
from actor;
```

```
-- you need to find the id number, first name, last name of an actor, of whom you  
know only the first name, 'joe'. what is one query would you use to obtain this  
information.
```

```
select actor_id, first_name, last_name  
from actor  
where first_name='joe';
```

```
-- find all actors whose last name contain the letter GEN
```

```
select first_name, last_name  
from actor  
where last_name like '%gen%';
```

```
-- find all actors whose last names contain the letter LI. this time, order the rows  
by last name and first name in that order
```

```
select last_name, first_name  
from actor  
where last_name like '%LI%'  
order by last_name,first_name;
```

```
-- using IN, display the country id and country columns of the following  
countries; afghanistan,bangladesh,china.
```

```
select country,country_id  
from country  
where country in ('afghanistan','bangladesh','china');
```

```
-- list the last names of actors, as well as how many actors have that last name
```

```
select last_name, count(last_name) as 'actors with same last name'  
from actor  
group by last_name;
```

```
-- list last names of actors and the number of actors who have the last name but  
only for names that are shared by at least two actors.
```

```
select last_name, count(last_name)  
from actor  
group by last_name  
having count(last_name) >'2' order by count(last_name) desc;
```

```
***** END *****
```